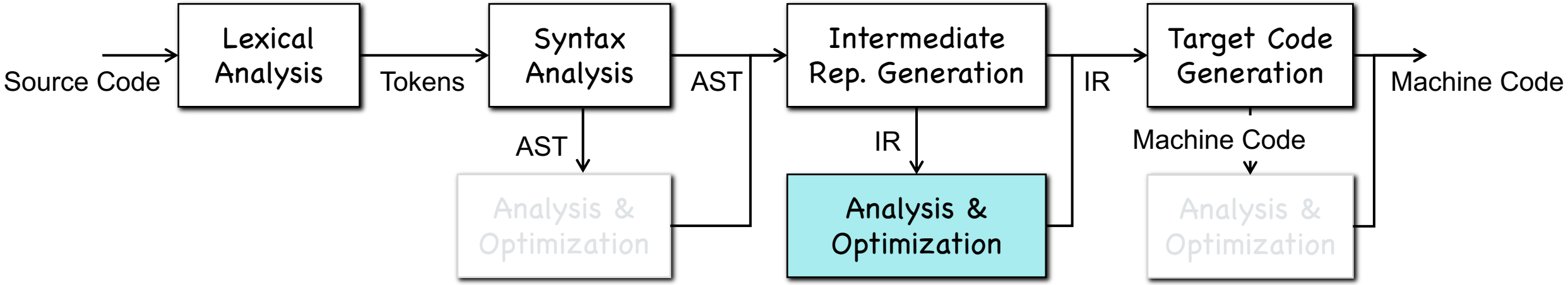
Made by Qingkai Shi
qingkaishi@nju.edu.cn
南京大学编译技术
及软件安全研究组

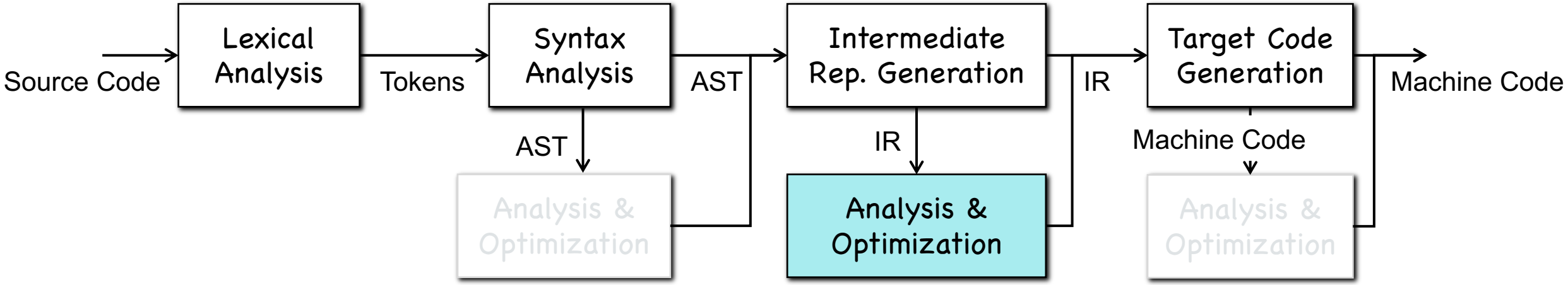
Chapter 12-1
Inter-procedural Analysis

Data Flow Analysis (DFA)



- How do we define a DFA?
- What are flow-insensitive, flow-sensitive, and path-sensitive analysis?
- What are symbolic execution and concolic execution?

Inter-procedural DFA



- Why inter-procedural DFA?
  - virtual method invocation
  - detection of software defects
  - …

```
public void foo(List l, int x) { l.add(x); }
```

MySQL

Bug #87203, which is detected by our work

---

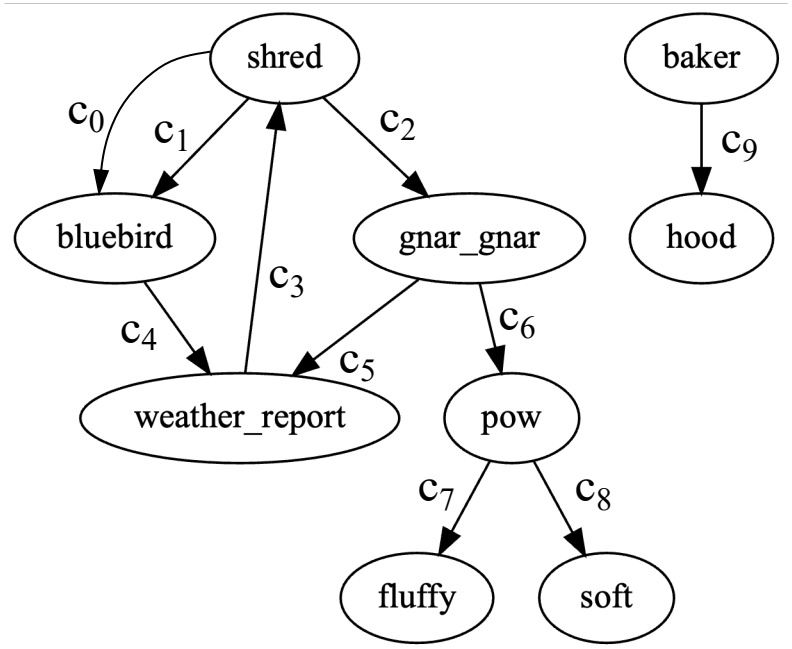
36 functions over 11 compiling units

PART I: Basics

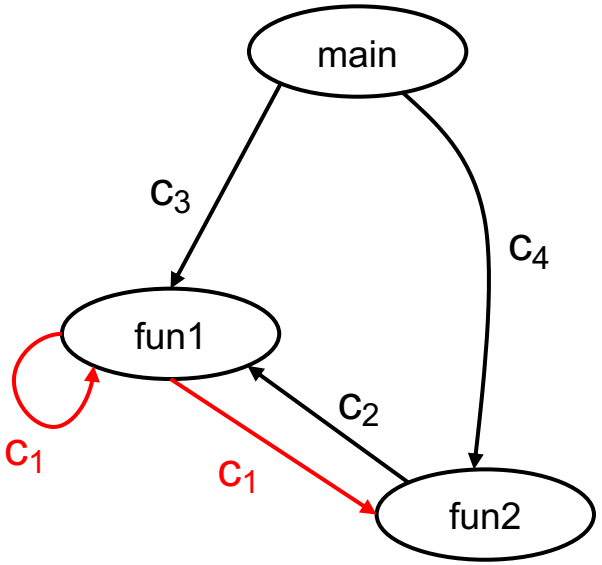
Call Graph (CG)

A bit different from the definition in the dragon book.

- Nodes --- Functions
- Edges --- Call relations between functions
- Edge Labels --- Call sites



Call Chains



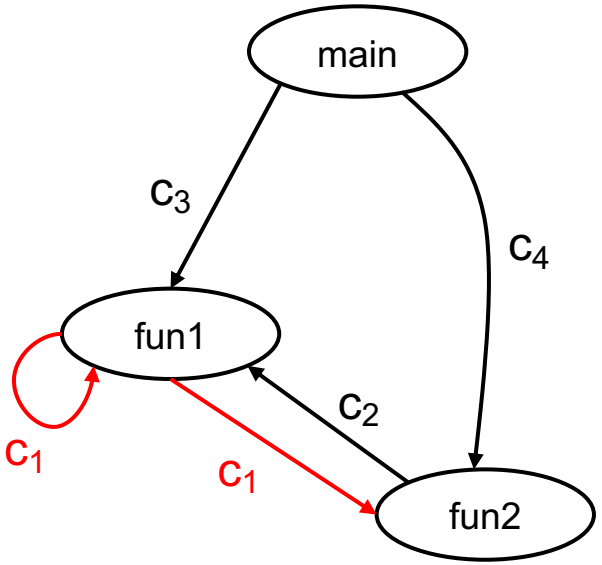
Call chains:

- c_3-c_1-…
- c_4-c_2-c_1-…
- …

```
          int (*pf)(int);

          int fun1(int x) {
              if (x < 10)
c1:               return (*pf)(x+1);
              else
                  return x;
          }

          int fun2(int y) {
              pf = &fun1;
c2:           return (*pf)(y);
          }

          void main() {
c3:           pf = &fun2; fun1(1);
c4:           fun2(5)
          }
```

It may call different functions based on different calling contexts
--- call chains (CG paths)!

Context Sensitivity



Call chains:

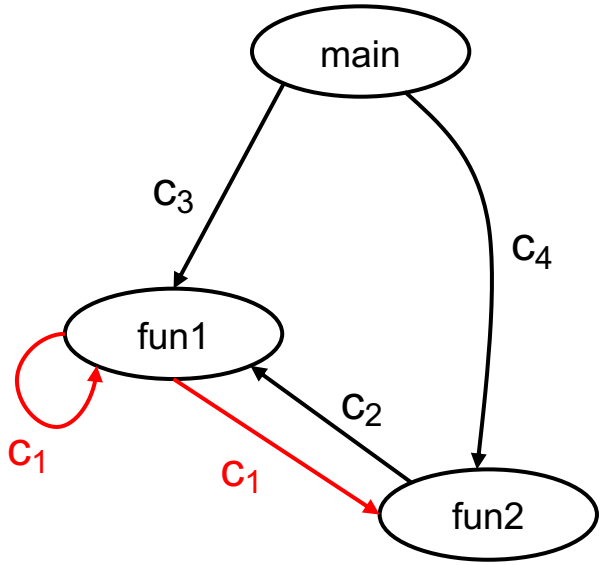
- c_3-c_1-…
- c_4-c_2-c_1-…
- …

```
            int (*pf)(int);

            int fun1(int x) {
                if (x < 10)
c1:                 return (*pf)(x+1);
                else
                    return x;
            }

            int fun2(int y) {
                pf = &fun1;
c2:             return (*pf)(y);
            }

            void main() {
c3:             pf = &fun2; fun1(1);
c4:             fun2(5)
            }
```

An analysis is context-insensitive, it can only produce:
fun_1 may call fun_1 or fun_2

If an analysis can answer in what call chains, fun_1 calls fun_1,
we say the analysis is context-sensitive!

Context Sensitivity



Call chains:

- c_3-c_1-…
- c_4-c_2-c_1-…
- …

```
          int (*pf)(int);

          int fun1(int x) {
              if (x < 10)
c1:               return (*pf)(x+1);
              else
                  return x;
          }

          int fun2(int y) {
              pf = &fun1;
c2:           return (*pf)(y);
          }

          void main() {
c3:           pf = &fun2; fun1(1);
c4:           fun2(5)
          }
```

To distinguish different function behaviors at different contexts

Recap:

- Flow-sensitivity
- Path-sensitivity

Context-Sensitive DFA

- **Clone-based** interprocedural DFA
- **Summary-based** interprocedural DFA

Clone-Based Analys

```
        for (i = 0; i < n; i++) {
c1:             t1 = g(0);
c2:             t2 = g(243);
c3:             t3 = g(243);
                X[i] = t1+t2+t3;
        }

        int g (int v) {
c4:         return f(v);
        }

        int f (int v) {
            return (v+1);
        }
```

clone

```
        for (i = 0; i < n; i++) {
c1:             t1 = g1(0);
c2:             t2 = g2(243);
c3:             t3 = g3(243);
                X[i] =  t1+t2+t3;
        }

        int g1 (int v) {
c4.1:           return f1(v);
        }

        int g2 (int v) {
c4.2:           return f2(v);
        }

        int g3 (int v) {
c4.3:           return f3(v);
        }

        int f1 (int v) {
            return (v+1);
        }

        int f2 (int v) {
            return (v+1);
        }

        int f3 (int v) {
            return (v+1);
        }
```

Context-Sensitive DFA

- **Clone-based** inter-procedural DFA
  - Let every function have only a single calling context
- **Summary-based** inter-procedural DFA
  - Top-down (TD) analysis: Analyze caller before callee
    - More specific to calling context
  - Bottom-up (BU) analysis: Analyze callee before caller
    - (i) Should assume all possible calling contexts; (ii) Easy to parallelize

TD Summary-Based Analysis

```
        for (i = 0; i < n; i++) {
c1:           t1 = g(0);
c2:           t2 = g(243);
c3:           t3 = g(243);
              X[i] = t1+t2+t3;
        }

      int g (int v) {
c4:       return f(v);
      }

      int f (int v) {
          return (v+1);
      }
```

reuse

- ForLoop → g() → f()
- Summary of g():
  - 0 → 1
  - 243 → 244

BU Summary-Based Analysis

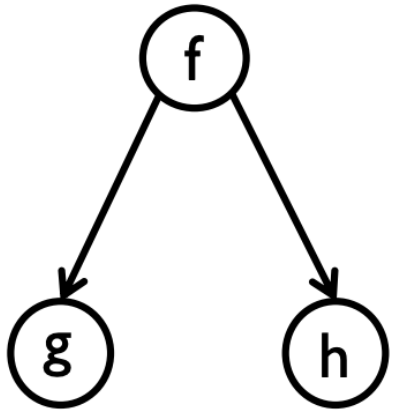
```
        for (i = 0; i < n; i++) {
c1:             t1 = g(0);
c2:             t2 = g(243);
c3:             t3 = g(243);
                X[i] = t1+t2+t3;
        }

        int g (int v) {
c4:         return f(v);
        }

        int f (int v) {
            return (v+1);
        }
```

- f() → g() → ForLoop

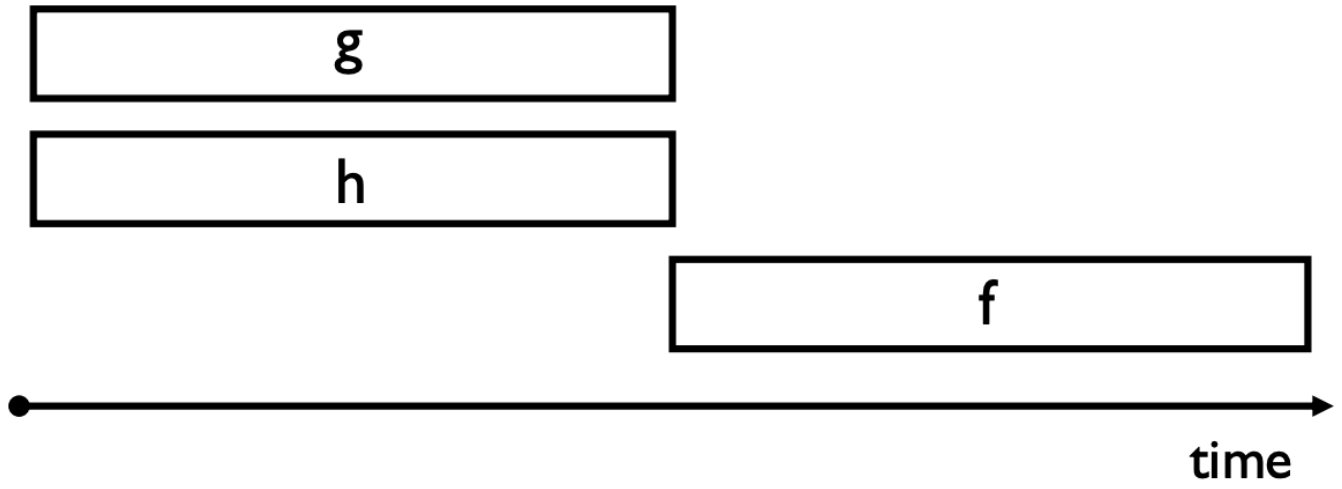
reuse

- Summary of f(): $x \longmapsto x + 1$
- Summary of g(): $x \longmapsto x + 1$

Parallelizing BU Analysis

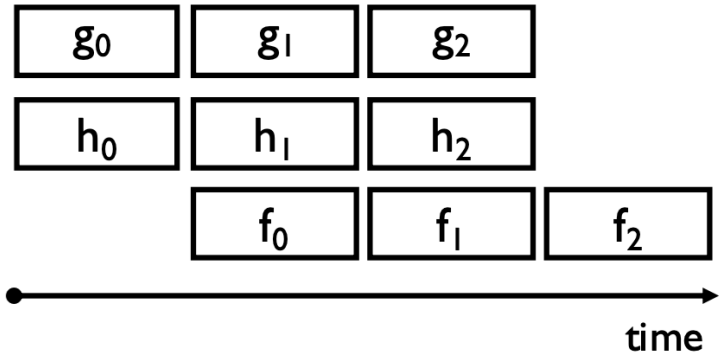


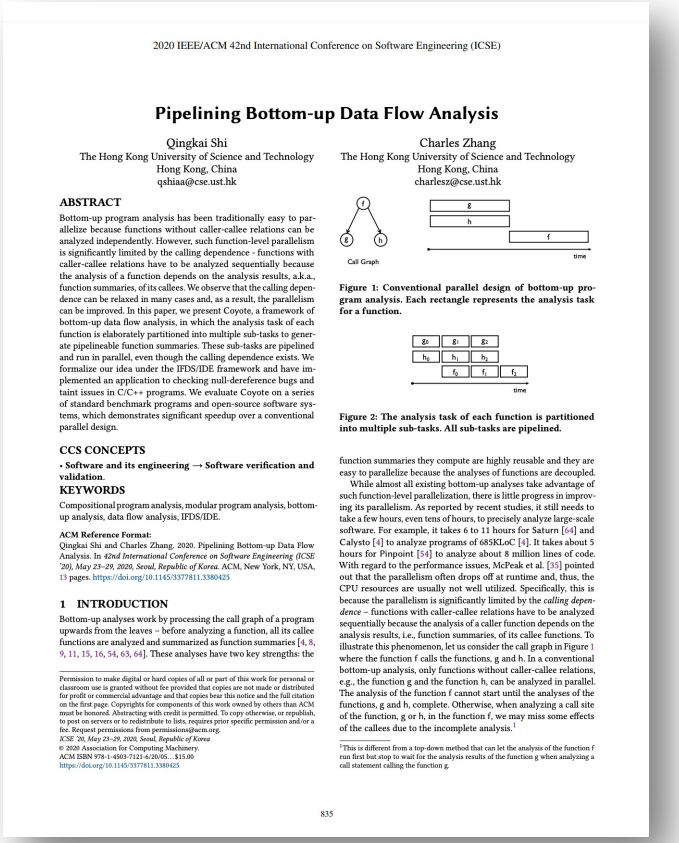
Call Graph



Pipelining BU Analysis

Pipelining Bottom-up Data Flow Analysis.
Qingkai Shi and Charles Zhang.
The ACM/IEEE International Conference on Software Engineering (ICSE'20).





2020 IEEE/ACM 42nd International Conference on Software Engineering (ICSE)

Pipelining Bottom-up Data Flow Analysis

Qingkai Shi
The Hong Kong University of Science and Technology
Hong Kong, China
qshiaa@cse.ust.hk

Charles Zhang
The Hong Kong University of Science and Technology
Hong Kong, China
charlesz@cse.ust.hk

ABSTRACT

Bottom-up program analysis has been traditionally easy to parallelize because functions without caller-callee relations can be analyzed independently. However, such function-level parallelism is significantly limited by the calling dependence - functions with caller-callee relations have to be analyzed sequentially because the analysis of a function depends on the analysis results, a.k.a., function summaries, of its callees. We observe that the calling dependence can be relaxed in many cases and, as a result, the parallelism can be improved. In this paper, we present Coyote, a framework of bottom-up data flow analysis, in which the analysis task of each function is elaborately partitioned into multiple sub-tasks to generate pipelineable function summaries. These sub-tasks are pipelined and run in parallel, even though the calling dependence exists. We formalize our idea under the IFDS/IDE framework and have implemented an application to checking null-dereference bugs and taint issues in C/C++ programs. We evaluate Coyote on a series of standard benchmark programs and open-source software systems, which demonstrates significant speedup over a conventional parallel design.

CCS CONCEPTS

• **Software and its engineering → Software verification and validation.**

KEYWORDS

Compositional program analysis, modular program analysis, bottom-up analysis, data flow analysis, IFDS/IDE.

ACM Reference Format:
Qingkai Shi and Charles Zhang. 2020. Pipelining Bottom-up Data Flow Analysis. In *42nd International Conference on Software Engineering (ICSE '20), May 23–29, 2020, Seoul, Republic of Korea.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3377811.3380425

1 INTRODUCTION

Bottom-up analyses work by processing the call graph of a program upwards from the leaves – before analyzing a function, all its callee functions are analyzed and summarized as function summaries [4, 8, 9, 11, 15, 16, 54, 63, 64]. These analyses have two key strengths: the function summaries they compute are highly reusable and they are easy to parallelize because the analyses of functions are decoupled.

While almost all existing bottom-up analyses take advantage of such function-level parallelization, there is little progress in improving its parallelism. As reported by recent studies, it still needs to take a few hours, even tens of hours, to precisely analyze large-scale software. For example, it takes 6 to 11 hours for Saturn [64] and Calysto [4] to analyze programs of 685KLoC [4]. It takes about 5 hours for Pinpoint [54] to analyze about 8 million lines of code. With regard to the performance issues, McPeak et al. [35] pointed out that the parallelism often drops off at runtime and, thus, the CPU resources are usually not well utilized. Specifically, this is because the parallelism is significantly limited by the *calling dependence* – functions with caller-callee relations have to be analyzed sequentially because the analysis of a caller function depends on the analysis results, i.e., function summaries, of its callee functions. To illustrate this phenomenon, let us consider the call graph in Figure 1 where the function f calls the functions, g and h. In a conventional bottom-up analysis, only functions without caller-callee relations, e.g., the function g and the function h, can be analyzed in parallel. The analysis of the function f cannot start until the analyses of the functions, g and h, complete. Otherwise, when analyzing a call site of the function, g or h, in the function f, we may miss some effects of the callees due to the incomplete analysis.[1]

Figure 1: Conventional parallel design of bottom-up program analysis. Each rectangle represents the analysis task for a function.

Figure 2: The analysis task of each function is partitioned into multiple sub-tasks. All sub-tasks are pipelined.

[1] This is different from a top-down method that can let the analysis of the function f run first but stop to wait for the analysis results of the function g when analyzing a call statement calling the function g.

ICSE '20, May 23–29, 2020, Seoul, Republic of Korea

ACM ISBN 978-1-4503-7121-6/20/05…$15.00
https://doi.org/10.1145/3377811.3380425

PART II: DFA as Graph Reachability

Recap: Constant Propagation

- Each variable v → UNDEF, Constant, NAC, denoted as m(v)

Transfer Function of x = y

- m(y) = Constant
- **=> m(x) = Constant**

- m(y) = UNDEF
- **=> m(x) = UNDEF**

- otherwise
- **=> m(x) = NAC**

Transfer Function of x = y + z

- m(y) = Constant, m(z) = Constant
- **=> m(x) = Constant**

- either of m(y) and m(z) is UNDEF
- **=> m(x) = UNDEF**

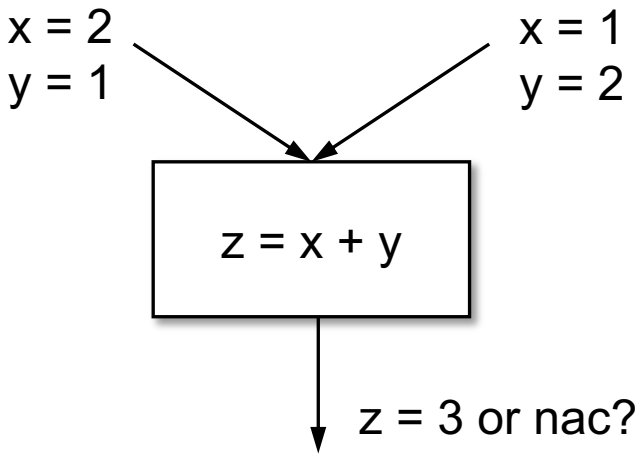
- otherwise
- **=> m(x) = NAC**

Recap: Constant Propagation

- Non-distributivity: Meet-Then-Transfer ≠ Transfer-Then-Meet

Merge Function: m(x) ∧ m(x')

- UNDEF ∧ Constant = Constant
- NAC ∧ Constant = NAC
- $Constant_1 \wedge Constant_1 = Constant_1$
- $Constant_1 \wedge Constant_2 = NAC$



Possibly Uninitialized Variables

- A statement uses a possibly uninitialized variable if there is a path to the statement without any definition of the variable.

Possibly Uninitialized Variables

- Each variable v → UNDEF, ~~Constant, NAC~~DEF, denoted as m(v)

Transfer Function of x = y

- m(y) = DEF
- **=> m(x) = DEF**

- m(y) = UNDEF
- **=> m(x) = UNDEF**

Transfer Function of x = y + z

- m(y) = DEF, m(z) = DEF
- **=> m(x) = DEF**

- either of m(y) and m(z) is UNDEF
- **=> m(x) = UNDEF**

Possibly Uninitialized Variables

- Each variable v → UNDEF, ~~Constant, NAC~~DEF, denoted as m(v)

Merge Function: m(x) ∧ m(x')

- UNDEF ∧ Constant = Constant
- NAC ∧ Constant = NAC
- $Constant_1 \wedge Constant_1 = Constant_1$
- $Constant_1 \wedge Constant_2 = NAC$

Constant Propagation

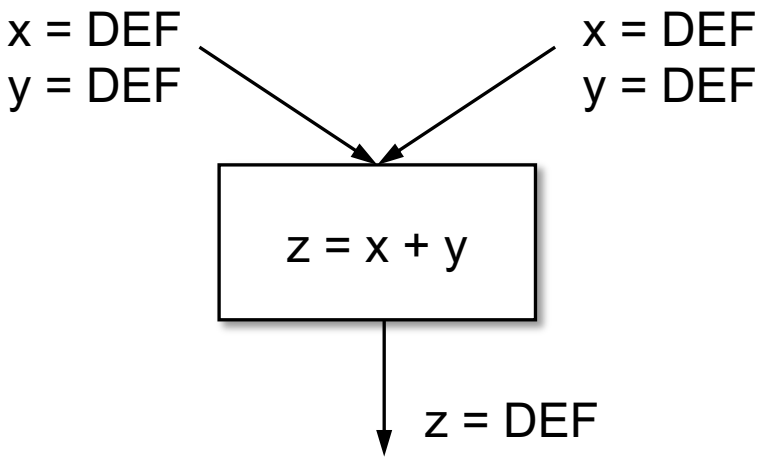
Merge Function: m(x) ∧ m(x')

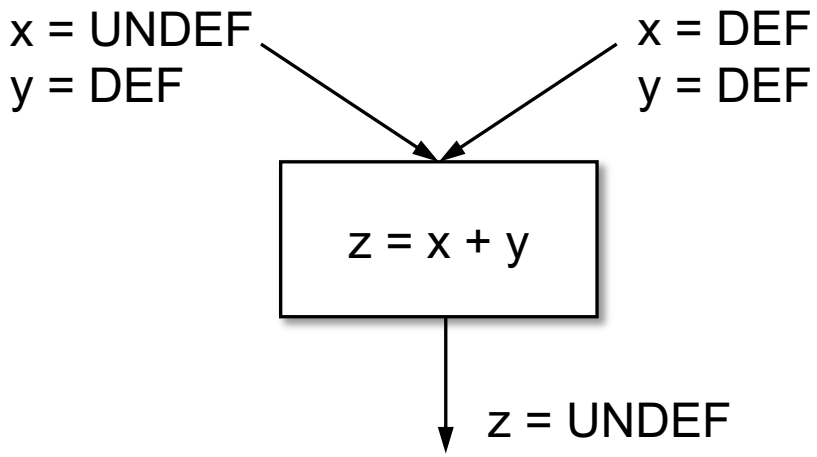
- UNDEF ∧ DEF = UNDEF
- DEF ∧ DEF = DEF
- DEF ∧ DEF = DEF
- DEF ∧ DEF = DEF

Uninitialized Variables

Possibly Uninitialized Variables

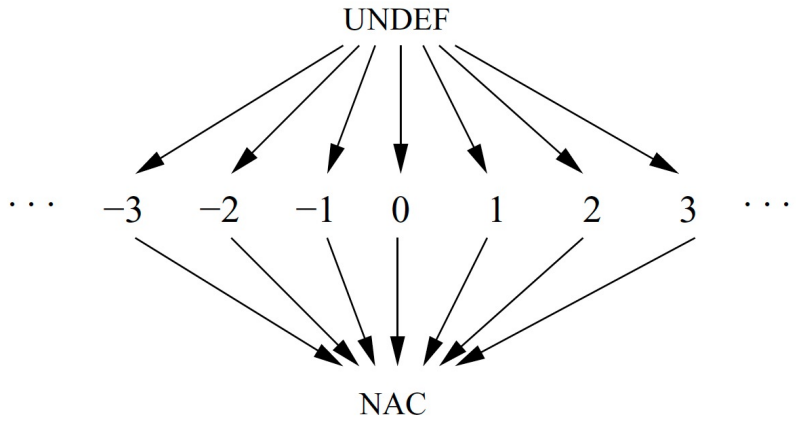
- Distributivity: f(x ∪ y) = f(x) ∪ f(y)



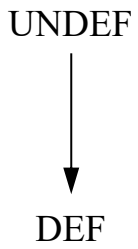


Possibly Uninitialized Variables

- Finity: Possible dataflow values are finite



Constant Propagation



Uninitialized Variables

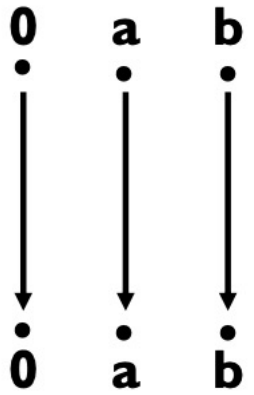
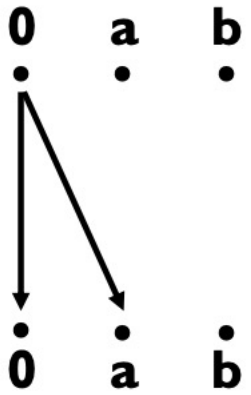
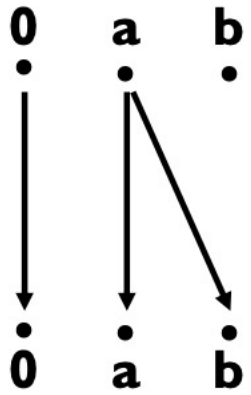
IFDS Problem

- Interprocedural Finite Distributive Subset Problem

Precise Inter-procedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995

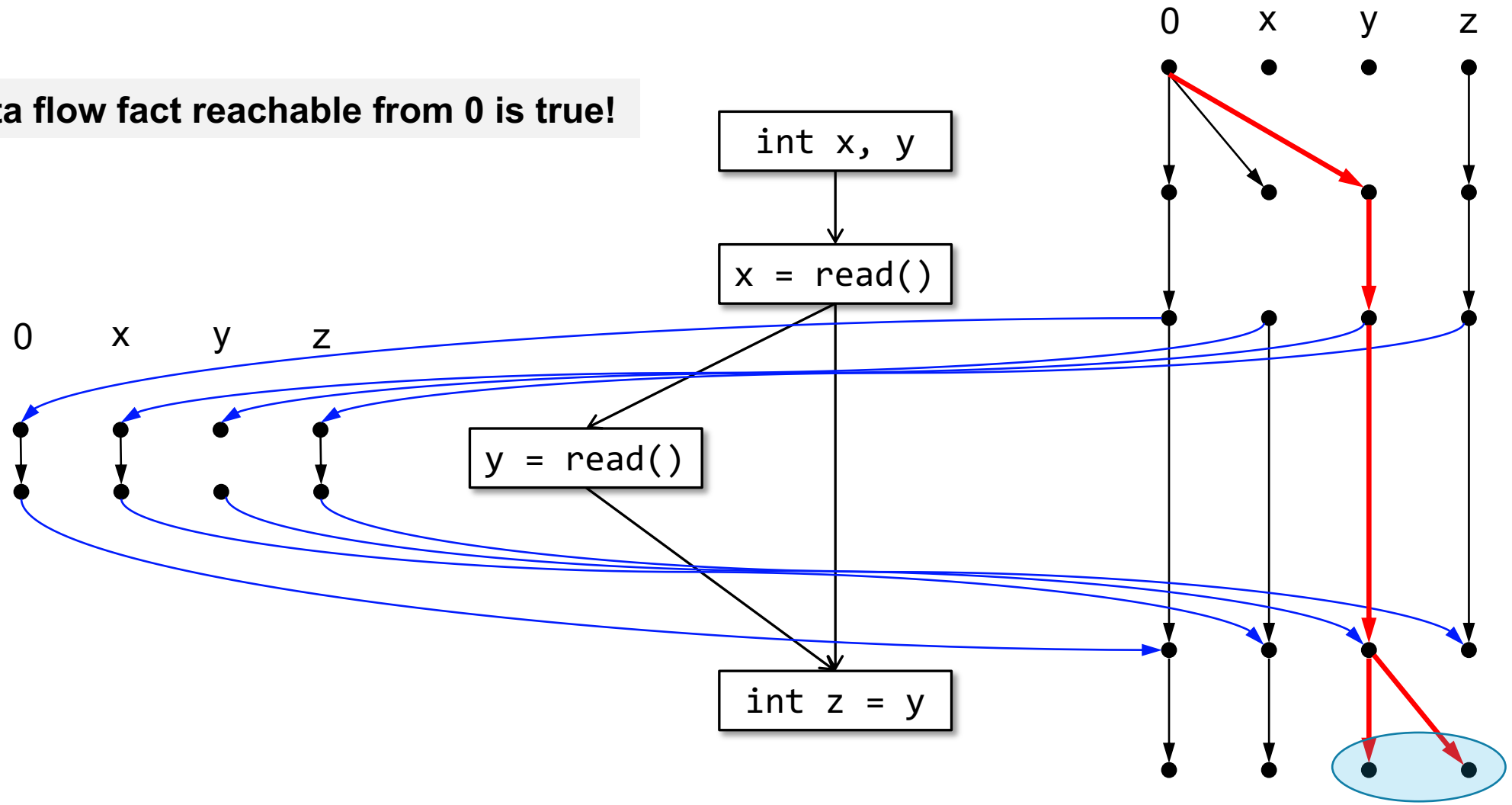


IFDS as Graph Reachability

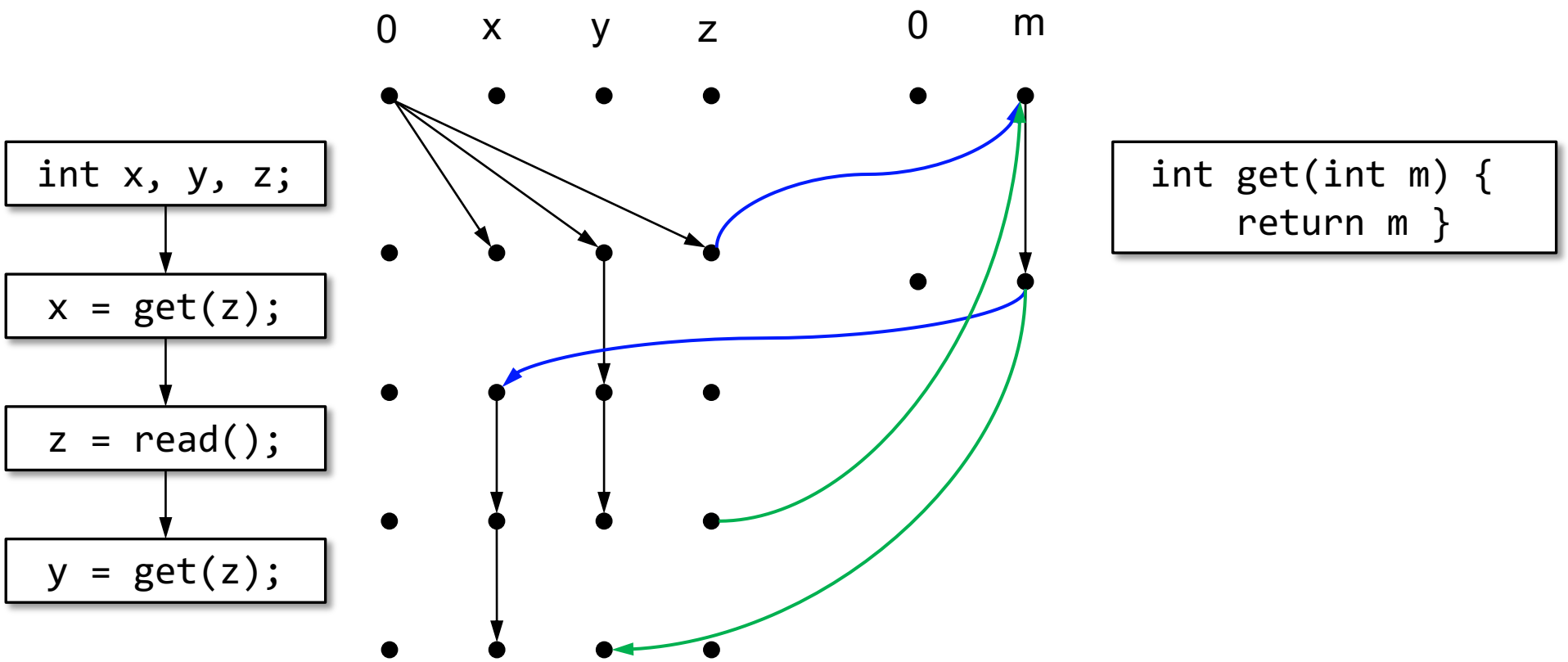
f(x) = x	f(x) = { a }	f(x) = x ∪ { b } if a∈x = x \ { b } otherwise
		

IFDS as Graph Reachability

Any data flow fact reachable from 0 is true!

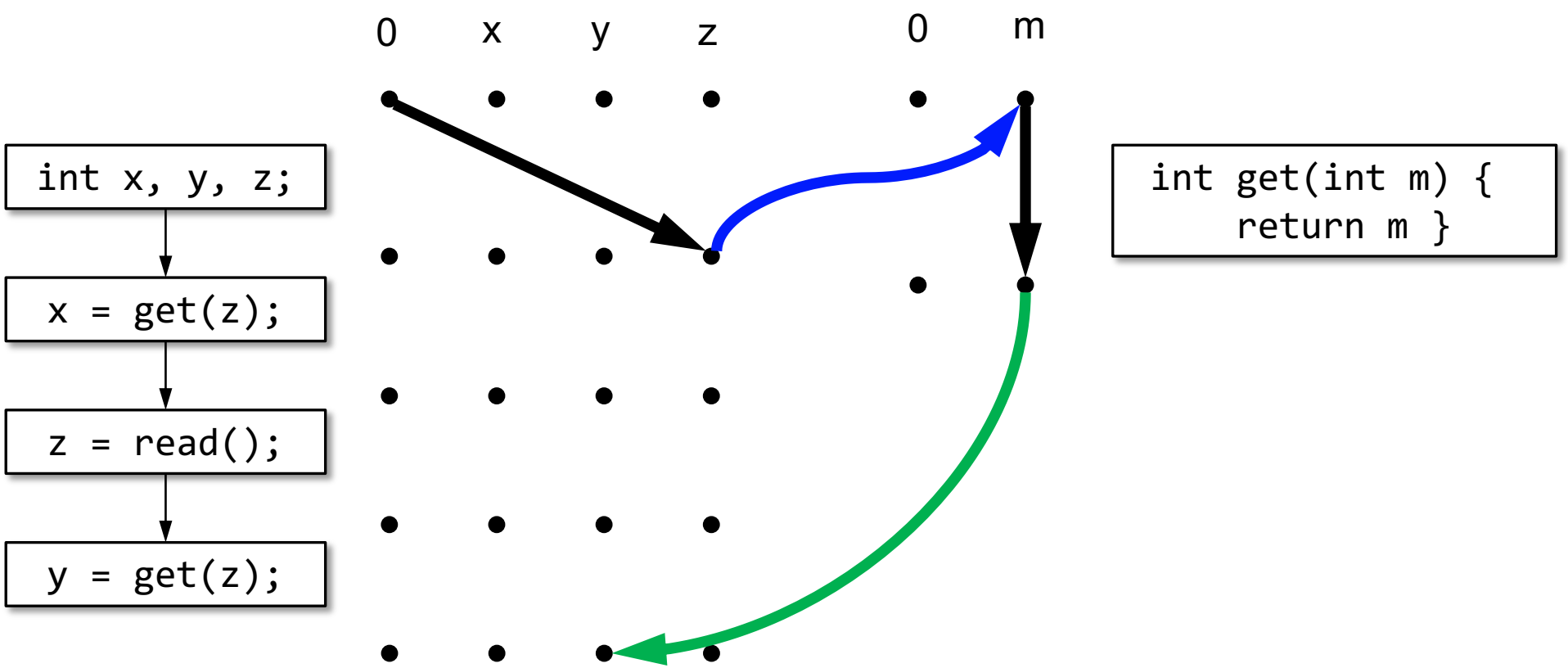


IFDS as Graph Reachability



PS: Edges between 0s are ommited

Context-Sensitivity

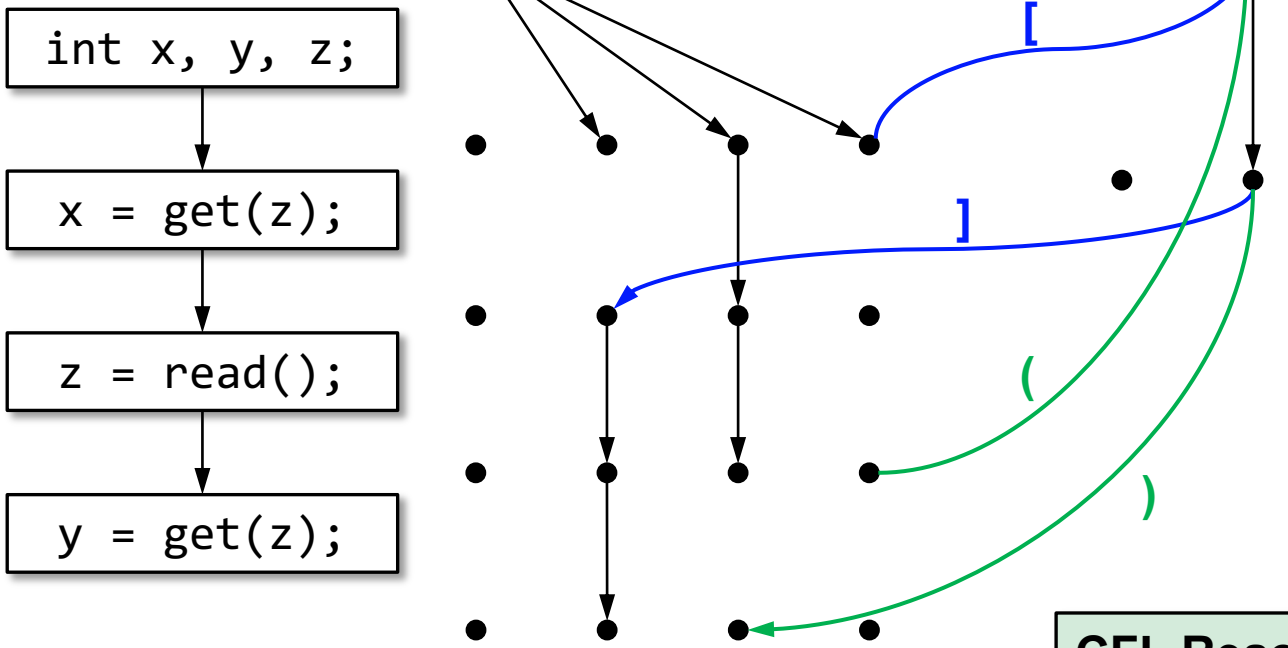


PS: Edges between 0s are ommited

y is uninitialized?

No! Why?

CFL Reachability




```
int get(int m) {
    return m }
```

Context-Free Grammar

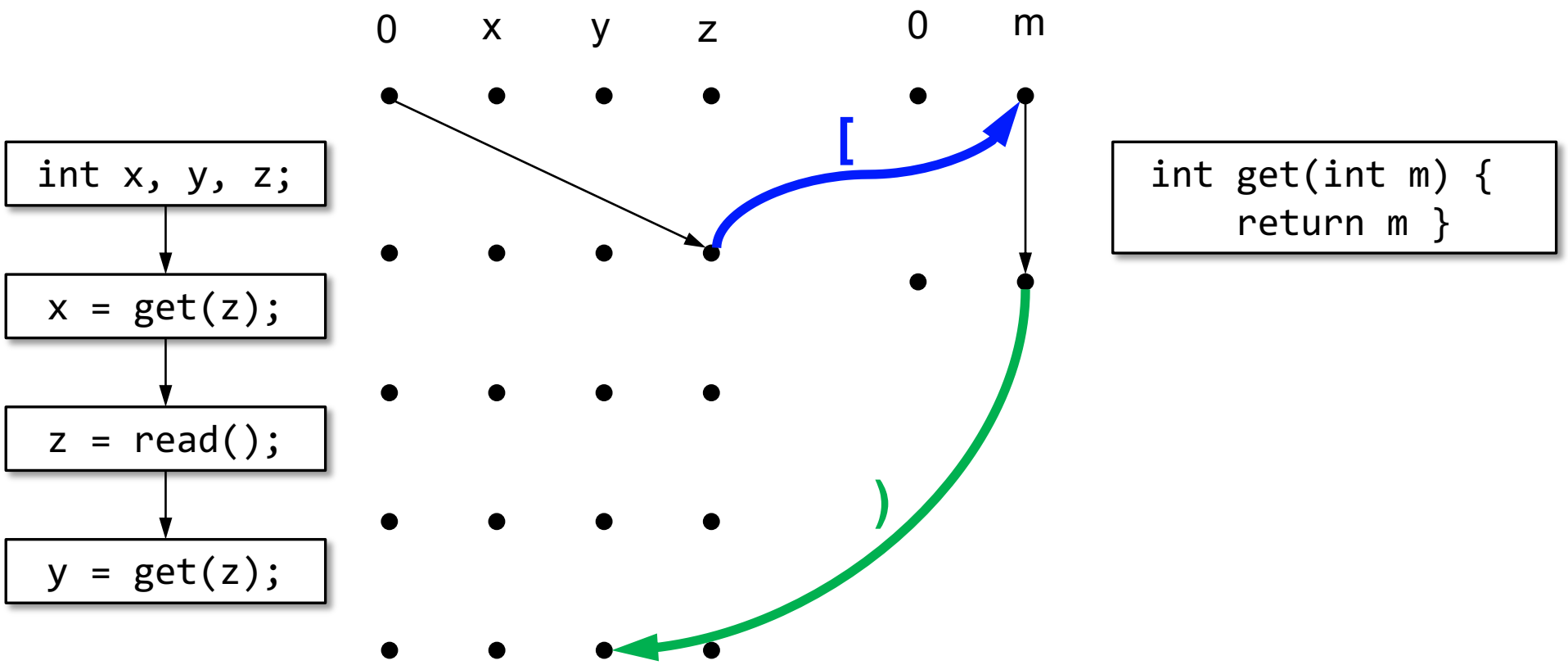
S → S S | (S) | [S] | ϵ

N → SN | (N | [N | ϵ

CFL Reachability:
There is a path from src to dst
The string of edge labels follows the grammar

PS: Edges between 0s are ommited

CFL Reachability



PS: Edges between 0s are ommited

y is initialized!

Computing Transitive Closure

- **Time**: $O(|V|^3/\log|V|)$; **Space**: $O(|V|^2)$

Subcubic algorithms for recursive state machines
Swarat Chaudhuri, POPL, 2008.

E: # edges

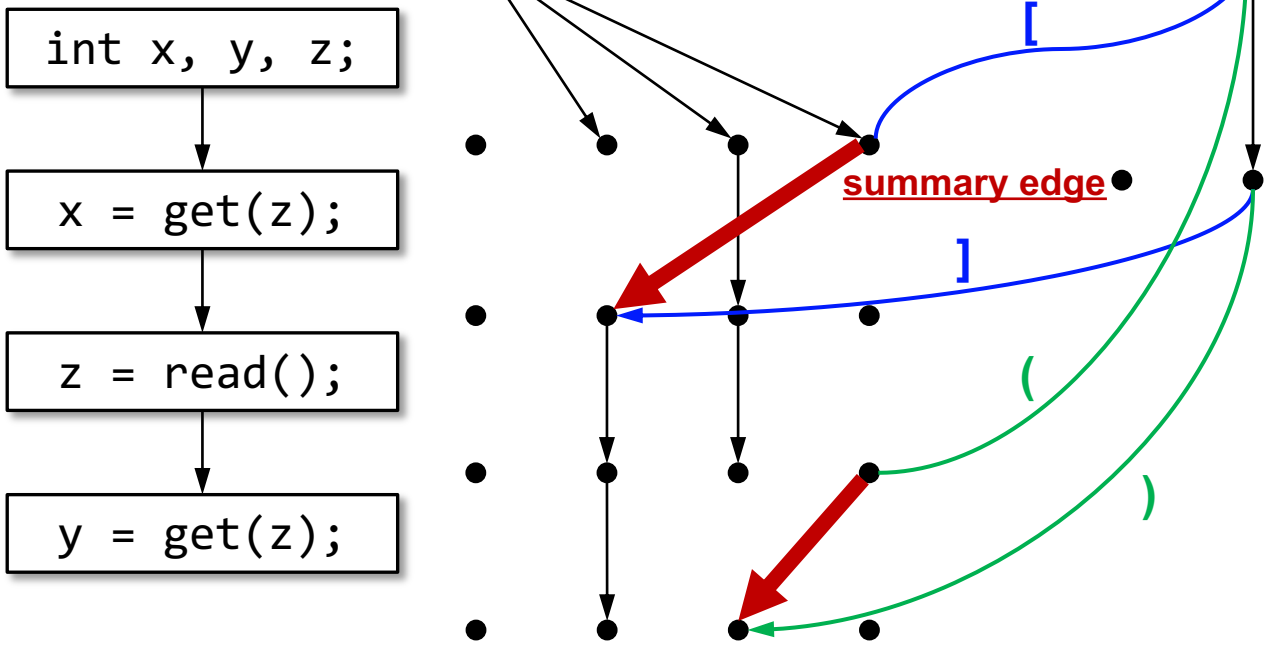
D: # dataflow facts

- A Tabulation Algorithm (**Time**: $O(|E||D|^3)$)

Precise Interprocedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995



Tabulation Algorithm




```
int get(int m) {
    return m }
```

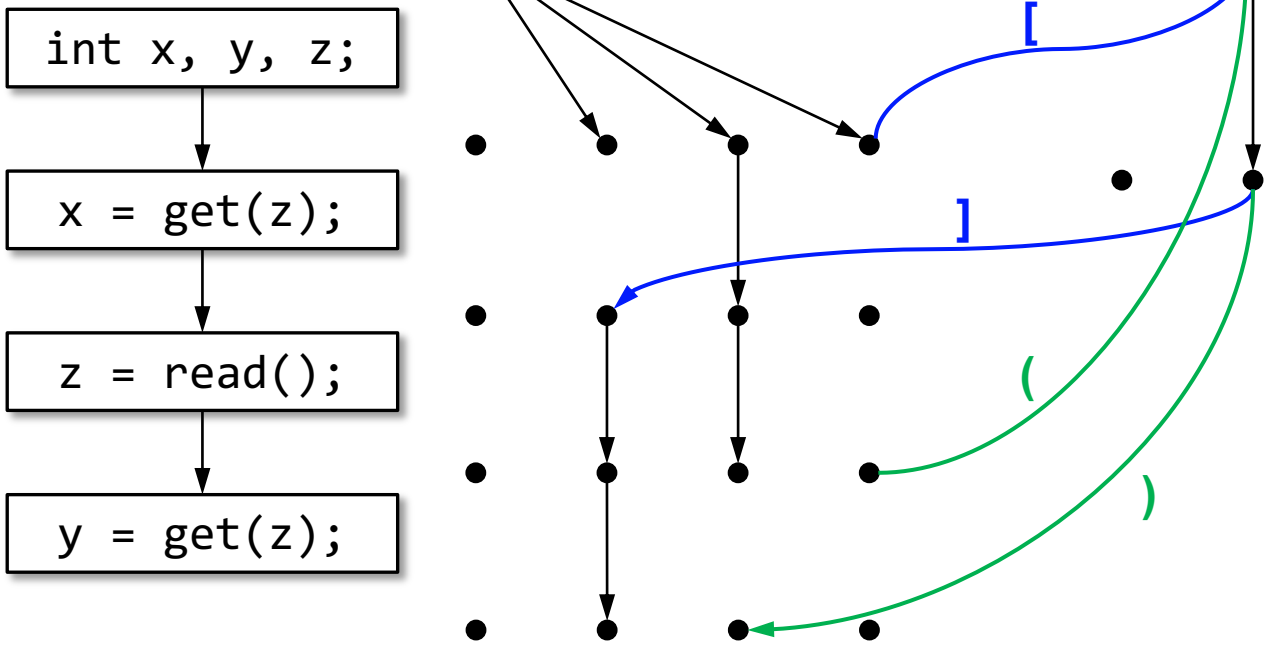
Basic Idea:
Avoid repeating visiting a function, via summary edge.

PS: Edges between 0s are ommited

Demand-Driven CFL-Reachability

- We do not always need a transitive closure.

Demand-Driven CFL-Reachability




```
int get(int m) {
    return m }
```

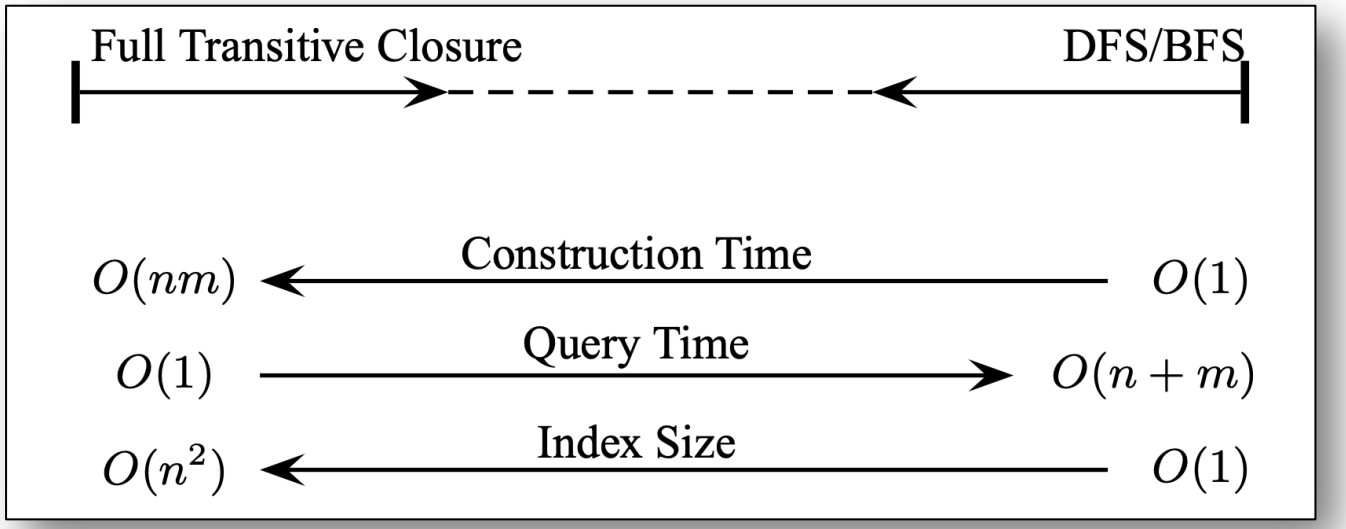
Basic Idea:
Search a path only for a query:
e.g.,
Is y initialized after y = get(z);

PS: Edges between 0s are ommited

Which Method is Better?

- Computing a transitive closure
  - O(1) query time;
  - ~ $O(n^3)$ preprocessing time --- may not finish;
- Search the graph for a path on demand
  - ~ O(n) query time; --- too long if we send queries frequently;
  - O(1) preprocessing;
- **Solution**: Indexing! An idea from the (graph) database community.

Indexing Conventional Reachability



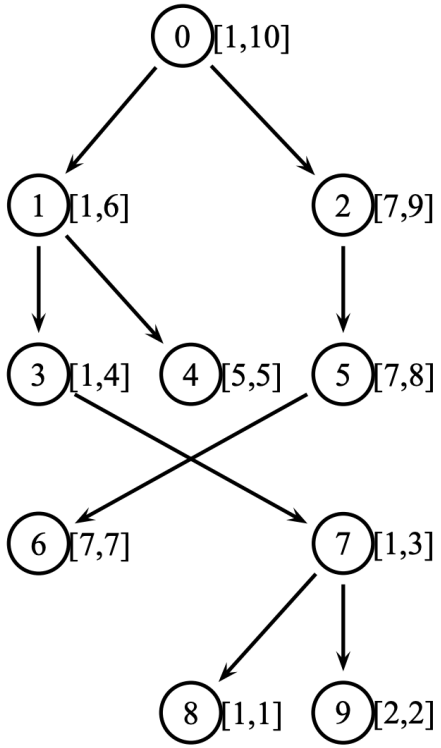
Work on DAG! Why?

Two types of indexing!

- I. Compression of transitive closure
- II. Pruned Search

Store some intermediate results to speed up reachability query

Indexing Conventional Reachability



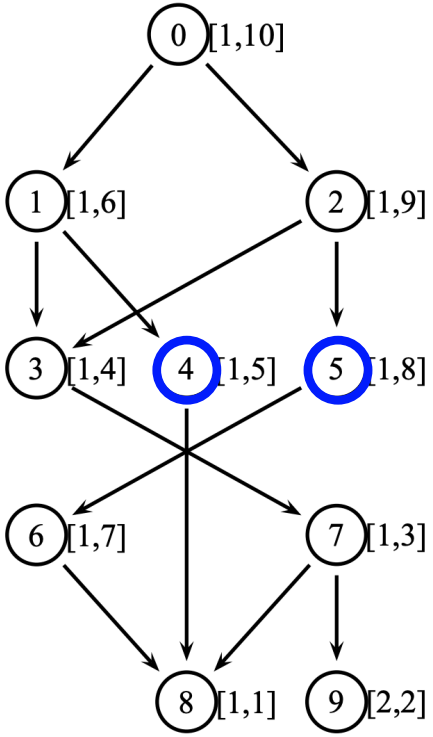
- **Label**: [low, high]
- high – post-order value
- low – minimum of the subtree

- $A \rightarrow B \Leftrightarrow Label_B \subseteq Label_A$

Generate O(n) intermediate results in O(n) to answer each query in O(1)

GRAIL: Scalable Reachability Index for Large Graphs
Hilmi Yıldırım, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

Indexing Conventional Reachability

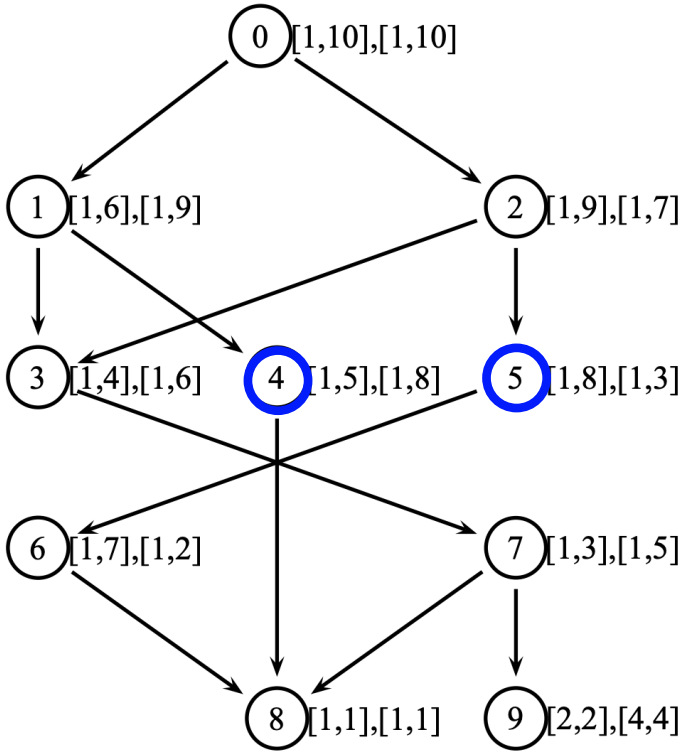


- **Label**: [low, high]
- high – post-order value
- low – minimum of the subgraph

- $A \rightarrow B \Rightarrow Label_B \subseteq Label_A$
- $Label_B \not\subseteq Label_A \Rightarrow A \not\rightarrow B$

GRAIL: Scalable Reachability Index for Large Graphs
Hilmi Yıldırım, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

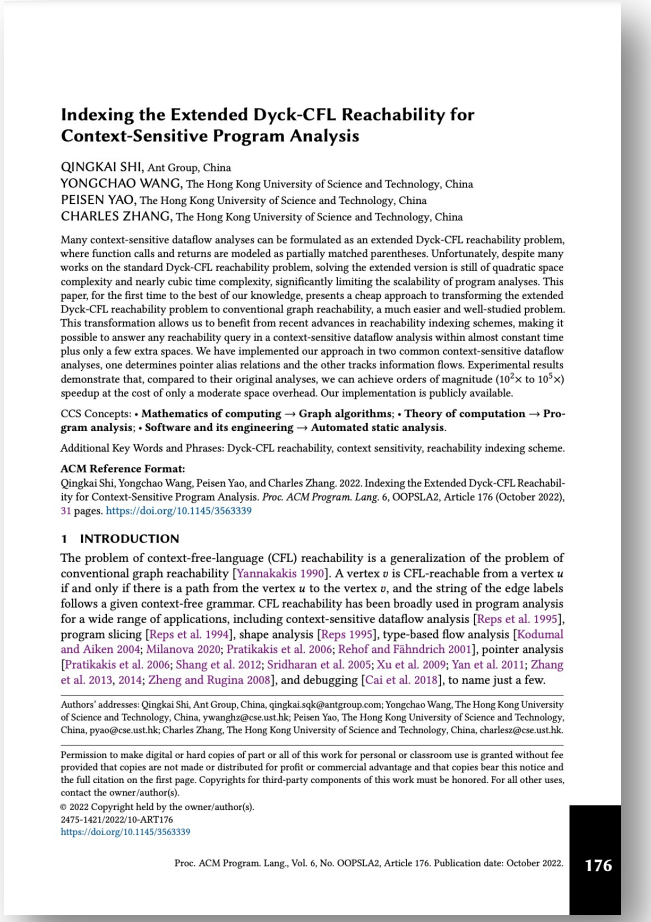
Indexing Conventional Reachability



- Generate multiple labels by randomly visiting the children of a node.
  - Node 4: **[1, 5] [1, 8]**
  - Node 5: **[1, 8] [1, 3]**
  - **[1, 8] $\not\subseteq$ [1, 5] $\Rightarrow$ 4 $\not\rightarrow$ 5**
  - **[1, 8] $\not\subseteq$ [1, 3] $\Rightarrow$ 5 $\not\rightarrow$ 4**
- **Q: How can the (un)reachability index help if there exists path between nodes?**

GRAIL: Scalable Reachability Index for Large Graphs
Hilmi Yıldırım, Vineet Chaoji, and Mohammed J. Zaki, VLDB 2010

Indexing CFL Reachability



Indexing the Extended Dyck-CFL Reachability for Context-Sensitive Program Analysis

QINGKAI SHI, Ant Group, China
YONGCHAO WANG, The Hong Kong University of Science and Technology, China
PEISEN YAO, The Hong Kong University of Science and Technology, China
CHARLES ZHANG, The Hong Kong University of Science and Technology, China

Many context-sensitive dataflow analyses can be formulated as an extended Dyck-CFL reachability problem, where function calls and returns are modeled as partially matched parentheses. Unfortunately, despite many works on the standard Dyck-CFL reachability problem, solving the extended version is still of quadratic space complexity and nearly cubic time complexity, significantly limiting the scalability of program analyses. This paper, for the first time to the best of our knowledge, presents a cheap approach to transforming the extended Dyck-CFL reachability problem to conventional graph reachability, a much easier and well-studied problem. This transformation allows us to benefit from recent advances in reachability indexing schemes, making it possible to answer any reachability query in a context-sensitive dataflow analysis within almost constant time plus only a few extra spaces. We have implemented our approach in two common context-sensitive dataflow analyses, one determines pointer alias relations and the other tracks information flows. Experimental results demonstrate that, compared to their original analyses, we can achieve orders of magnitude ($10^2\times$ to $10^5\times$) speedup at the cost of only a moderate space overhead. Our implementation is publicly available.

CCS Concepts: • **Mathematics of computing** → **Graph algorithms**; • **Theory of computation** → **Program analysis**; • **Software and its engineering** → **Automated static analysis**.

Additional Key Words and Phrases: Dyck-CFL reachability, context sensitivity, reachability indexing scheme.

ACM Reference Format:
Qingkai Shi, Yongchao Wang, Peisen Yao, and Charles Zhang. 2022. Indexing the Extended Dyck-CFL Reachability for Context-Sensitive Program Analysis. *Proc. ACM Program. Lang.* 6, OOPSLA2, Article 176 (October 2022), 31 pages. https://doi.org/10.1145/3563339

1 INTRODUCTION

The problem of context-free-language (CFL) reachability is a generalization of the problem of conventional graph reachability [Yannakakis 1990]. A vertex v is CFL-reachable from a vertex u if and only if there is a path from the vertex u to the vertex v, and the string of the edge labels follows a given context-free grammar. CFL reachability has been broadly used in program analysis for a wide range of applications, including context-sensitive dataflow analysis [Reps et al. 1995], program slicing [Reps et al. 1994], shape analysis [Reps 1995], type-based flow analysis [Kodumal and Aiken 2004; Milanova 2020; Pratikakis et al. 2006; Rehof and Fähndrich 2001], pointer analysis [Pratikakis et al. 2006; Shang et al. 2012; Sridharan et al. 2005; Xu et al. 2009; Yan et al. 2011; Zhang et al. 2013, 2014; Zheng and Rugina 2008], and debugging [Cai et al. 2018], to name just a few.

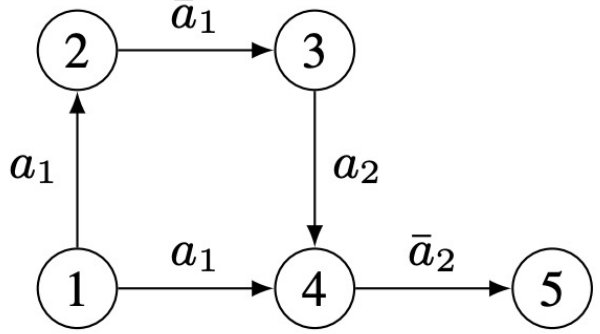
Authors' addresses: Qingkai Shi, Ant Group, China, qingkai.sqk@antgroup.com; Yongchao Wang, The Hong Kong University of Science and Technology, China, ywanghz@cse.ust.hk; Peisen Yao, The Hong Kong University of Science and Technology, China, pyao@cse.ust.hk; Charles Zhang, The Hong Kong University of Science and Technology, China, charlesz@cse.ust.hk.

2475-1421/2022/10-ART176
https://doi.org/10.1145/3563339

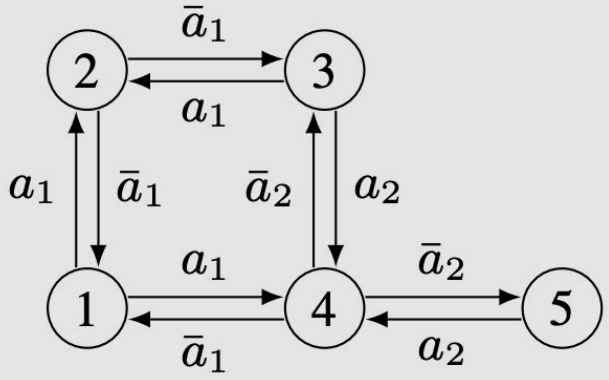
Indexing … CFL Reachability for Context-Sensitive Program Analysis
Qingkai Shi, Yongchao Wang, Peisen Yao, Charles Zhang
In Proceedings of the ACM on Programming Languages (OOPSLA), 2022
https://doi.org/10.1145/3563339

Beyond Context-Sensitivity

- CFL-reachability is often used for flow-insensitive alias analysis
- Alias analysis checks if two variables may have the same value



(a) The directed graph case.



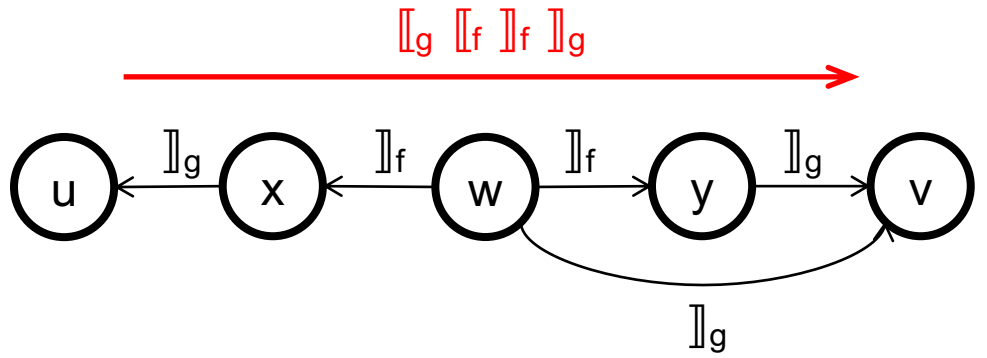
(b) The bidirected graph case.

$$S \rightarrow SS \mid a_1 S \bar{a}_1 \mid \ldots \mid a_k S \bar{a}_k \mid \varepsilon$$

Beyond Context-Sensitivity

- CFL-reachability is often used for flow-insensitive alias analysis
- We do not consider the statement order when building graph

```
x   =  w.f
y   =  w.f
u   =  x.g
v   =  y.g
w.g =   v
```



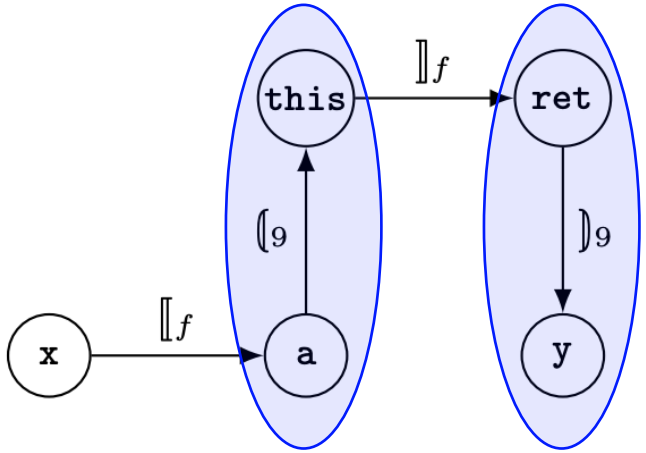
Context-Sensitive Alias Analysis?

- CFL-reachability can formulate context-sensitive analysis
- CFL-reachability can match field load and store in alias analysis
- How about a context-sensitive alias analysis?

Context-Sensitive Alias Analysis?

- Interleaved CFL Reachability!

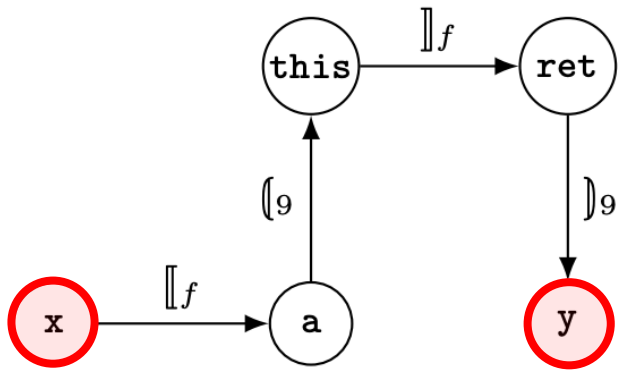
```
1:    class A {
2:      ...
3:      F getF() {
4:        return this.f;
5:      }
6:    }

7:    A a;  ...
8:    a.f = x;
9:    y = a.getF();
```

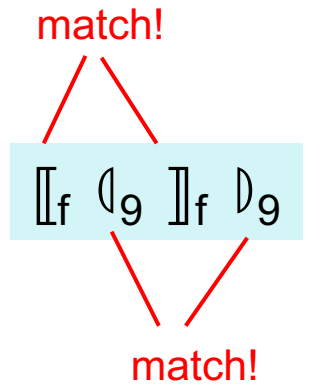


Context-Sensitive Alias Analysis?

- Interleaved CFL Reachability! **Undecidable!**

```
1:    class A {
2:      ...
3:      F getF() {
4:        return this.f;
5:      }
6:    }

7:    A a;  ...
8:    a.f = x;
9:    y = a.getF();
```





They belong to two different CFLs!

Thomas Reps. Undecidability of context-sensitive data-dependence analysis. TOPLAS 2000.
Qirun Zhang and Zhendong Su. Context-Sensitive Data-Dependence Analysis via Linear Conjunctive Language Reachability. POPL 2017.

PART III:
Fast CFL-Reachability for Context-Sensitive Data Dependence Analysis

Computing Transitive Closure

- **Time**: $O(|V|^3/\log|V|)$; **Space**: $O(|V|^2)$

Subcubic algorithms for recursive state machines
Swarat Chaudhuri, POPL, 2008.

- Tabulation Algorithm for IFDS (**Time**: $O(|E||D|^3)$)

Precise Interprocedural Dataflow Analysis via Graph Reachability
Reps, Horwitz, Sagiv, POPL 1995

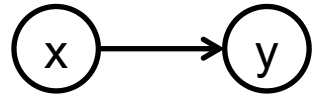


CFL-Reachability for program analysis may not be that hard, compared to general cases!

Data Dependence Analysis

- Which variable depends on which others
  - Same-level dependence (vars from same func)
  - Diff-level dependence (vas from diff func)
- Identify def-use chains in a program
- LHS depends on RHS

```
1  x ← 2
2  y ← x + 1
```



Your Turn!

```
1  result ← 0
2  for i ← 1 to n do
3      z ← x[i] · y[i]
4      result ← result + z
5  end
6  return result
```

: $h()$

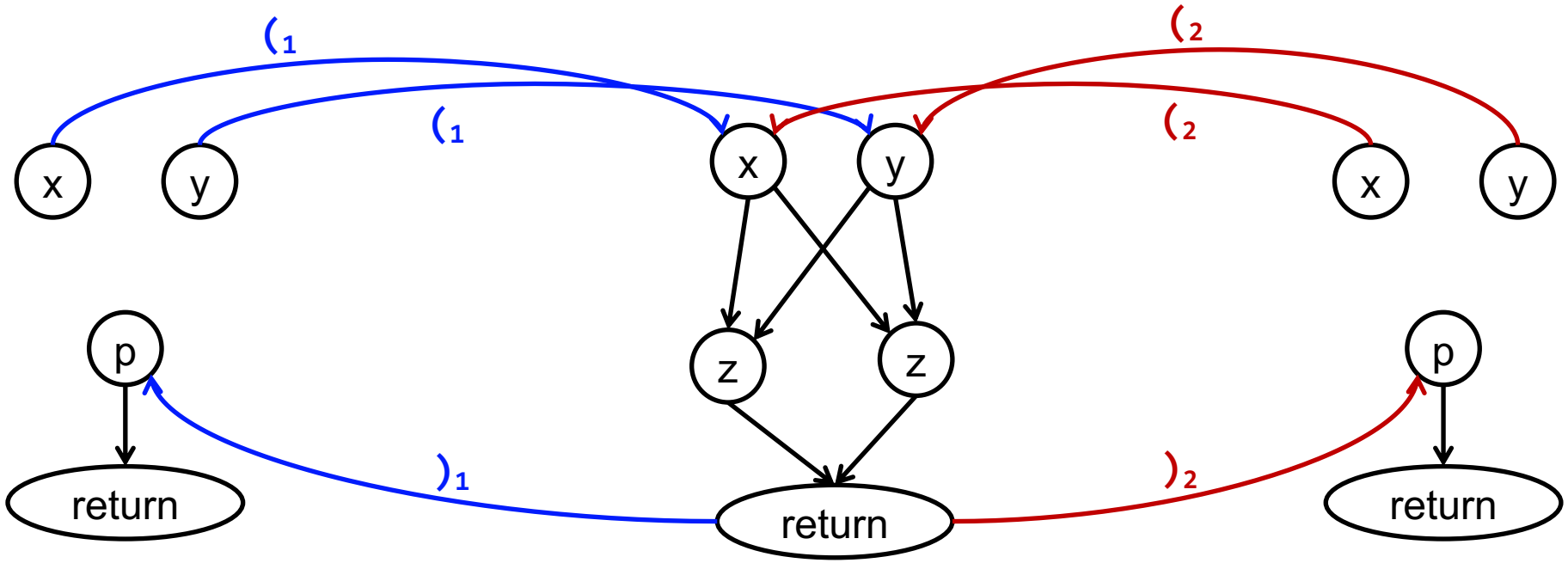
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

: $f(x, y)$

```
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

: $g()$

```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



: $h()$

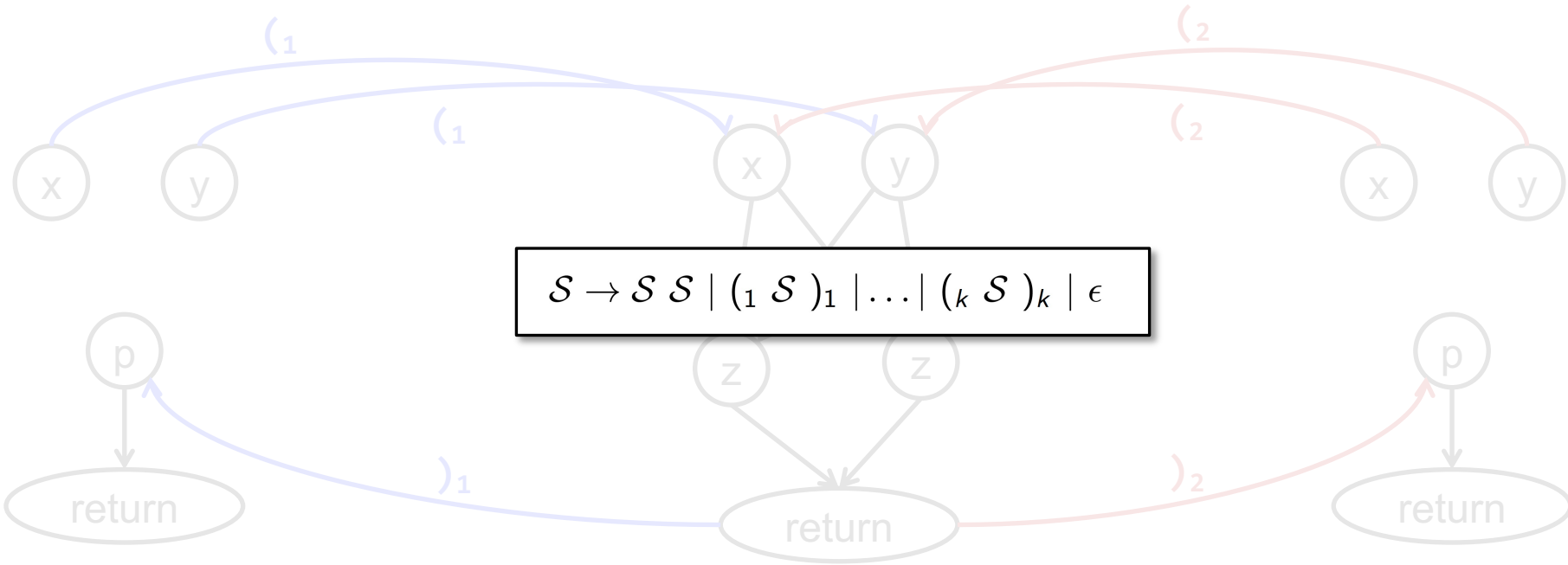
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

: $f(x, y)$

```
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

: $g()$

```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

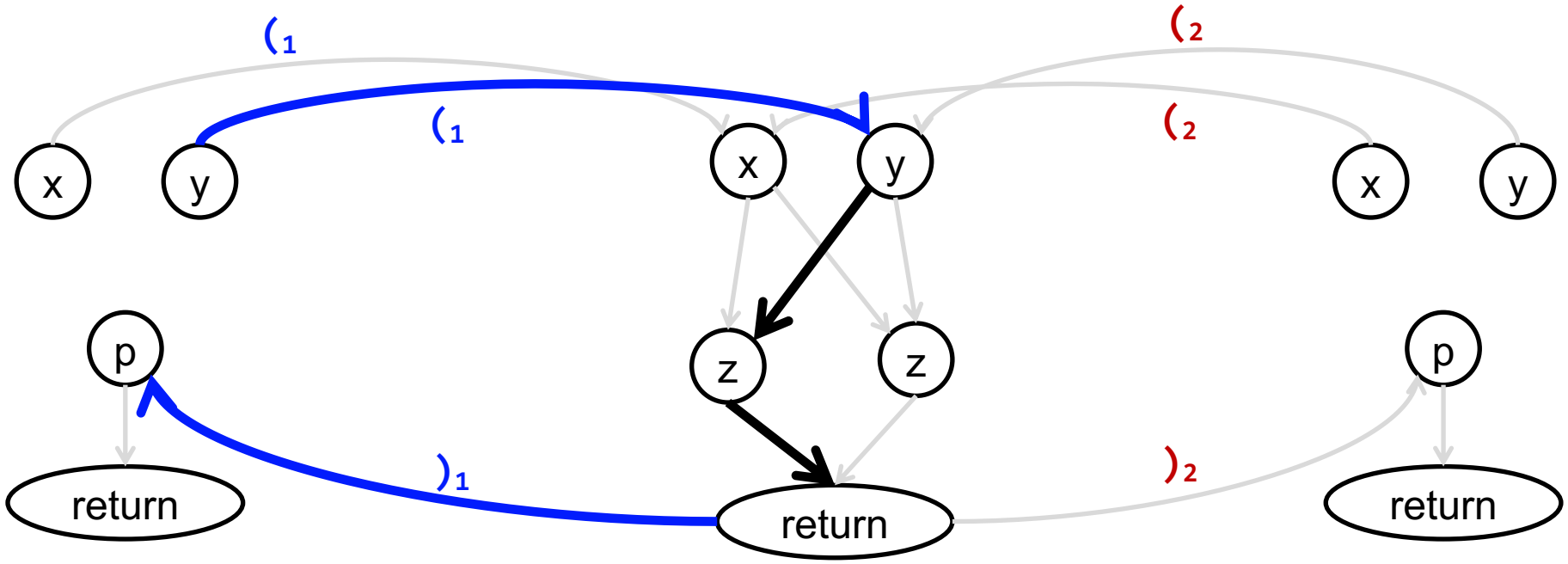


$$\mathcal{S} \to \mathcal{S}\ \mathcal{S} \mid (_1\ \mathcal{S}\ )_1 \mid \ldots \mid (_k\ \mathcal{S}\ )_k \mid \epsilon$$

```
: h()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

```
: f(x, y)
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

```
: g()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



: $h()$

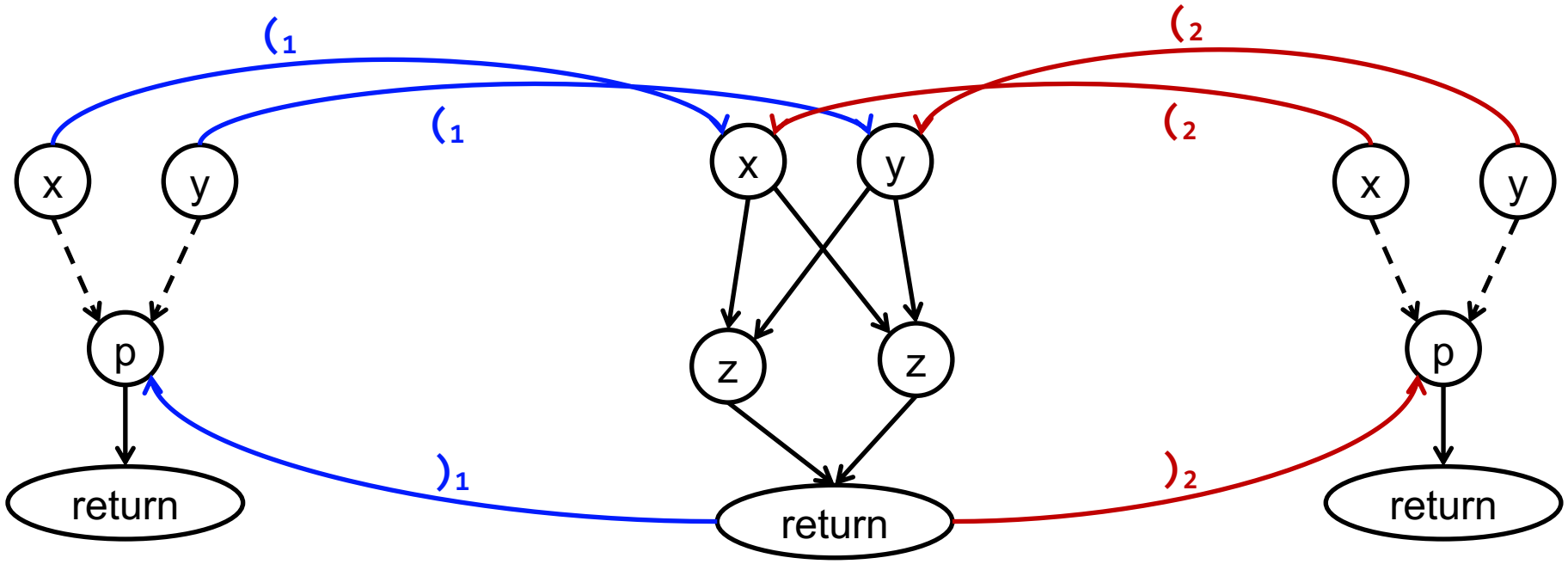
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

: $f(x, y)$

```
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

: $g()$

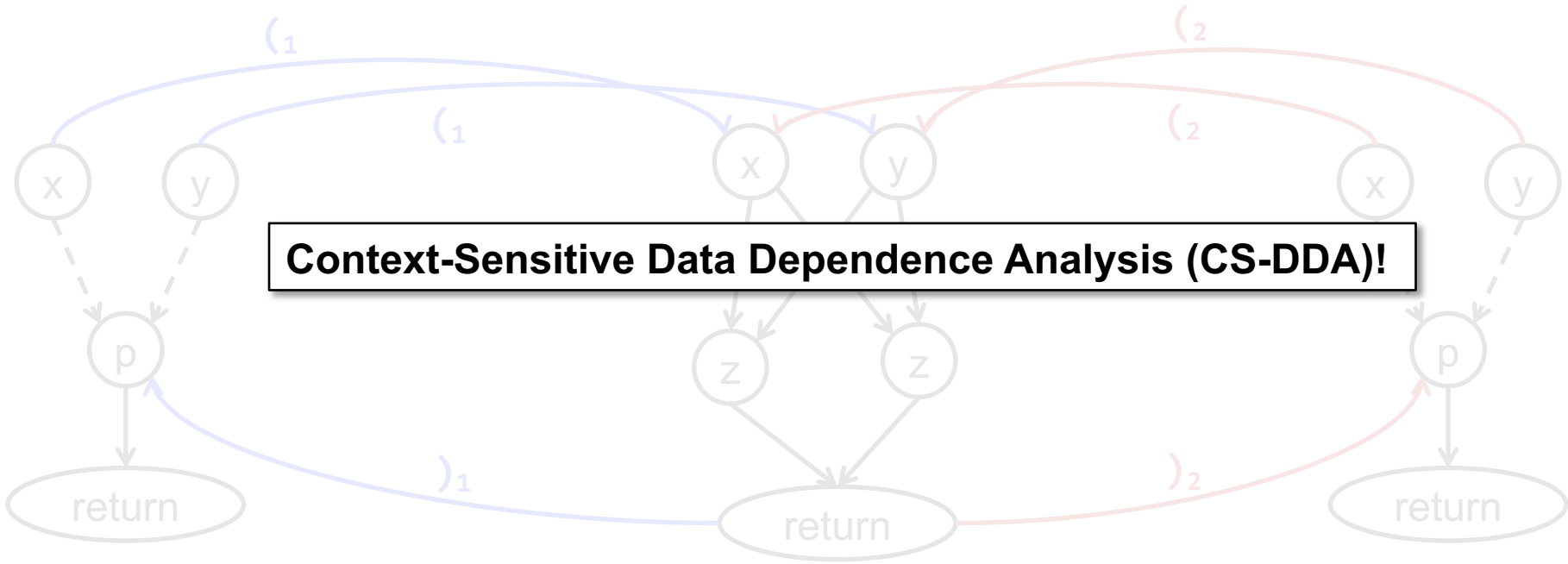
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



```
: h()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

```
: f(x, y)
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

```
: g()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



Context-Sensitive Data Dependence Analysis (CS-DDA)!

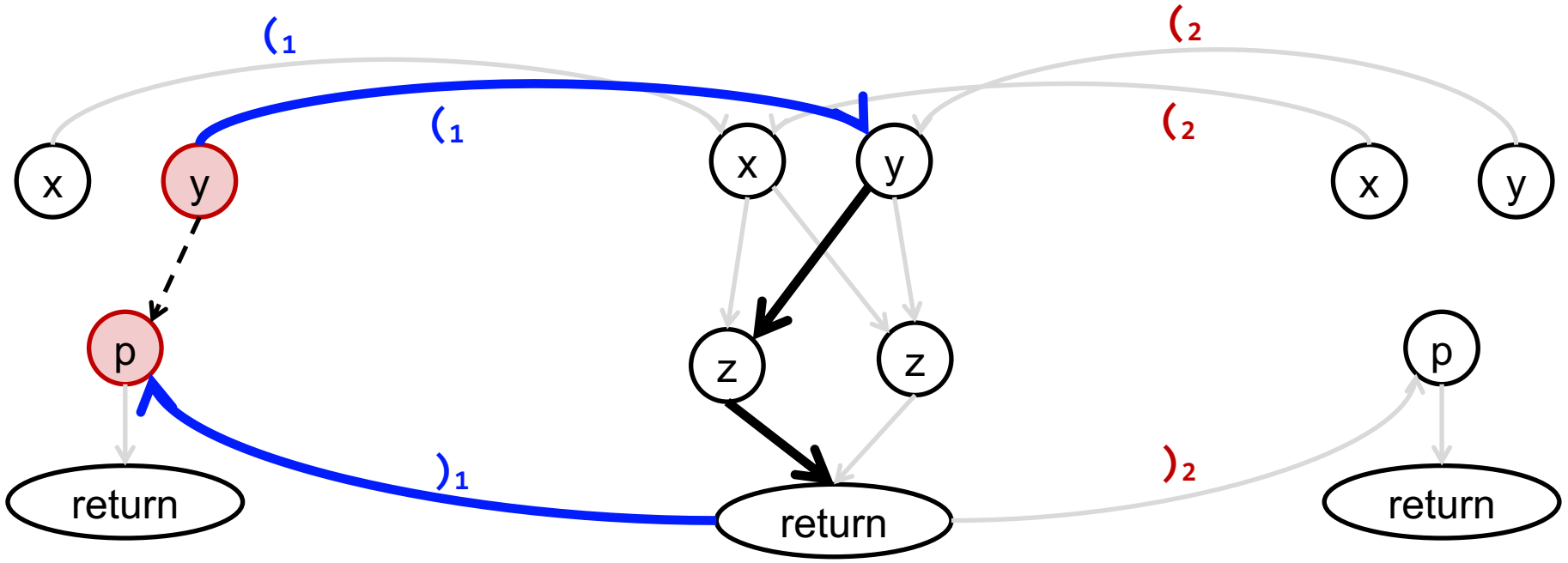
CS-DDA at the Same Level

- **Goal**: Answer any same-level data dependence query in O(1) after ~O(n) preprocessing

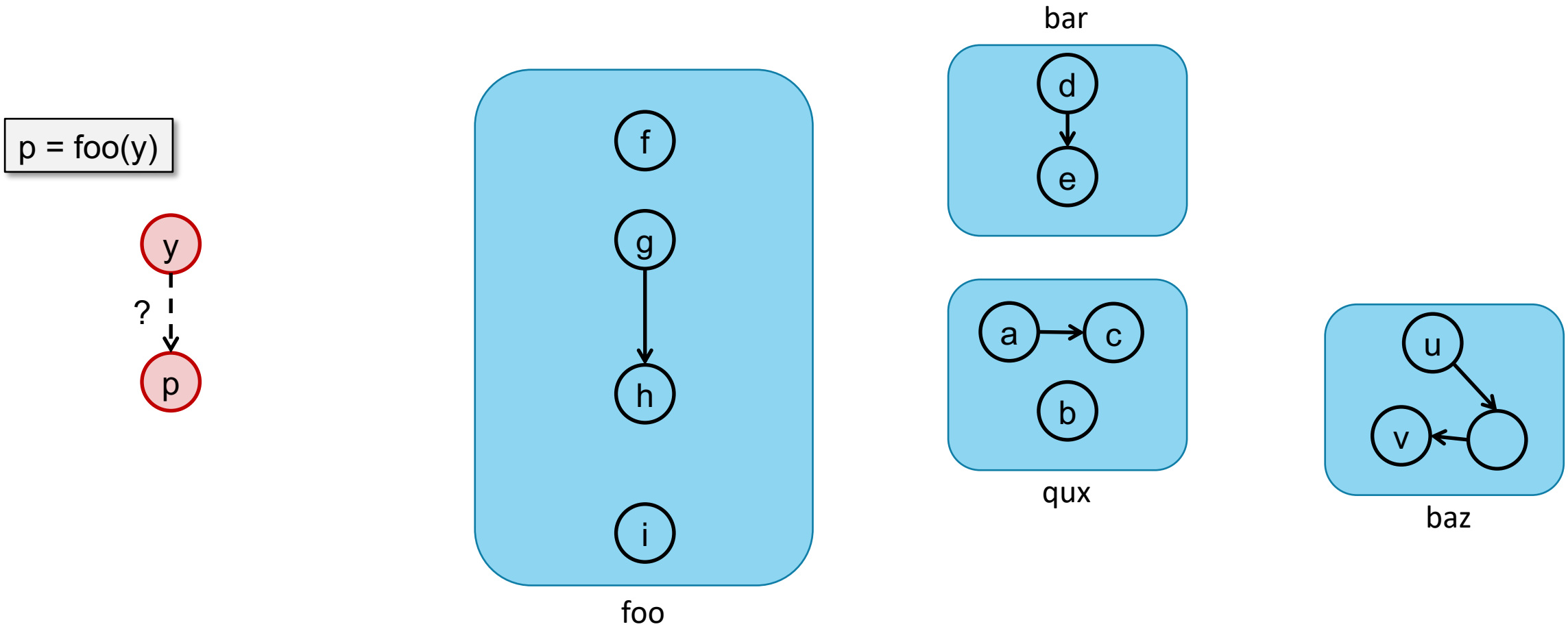
```
: h()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

```
: f(x, y)
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

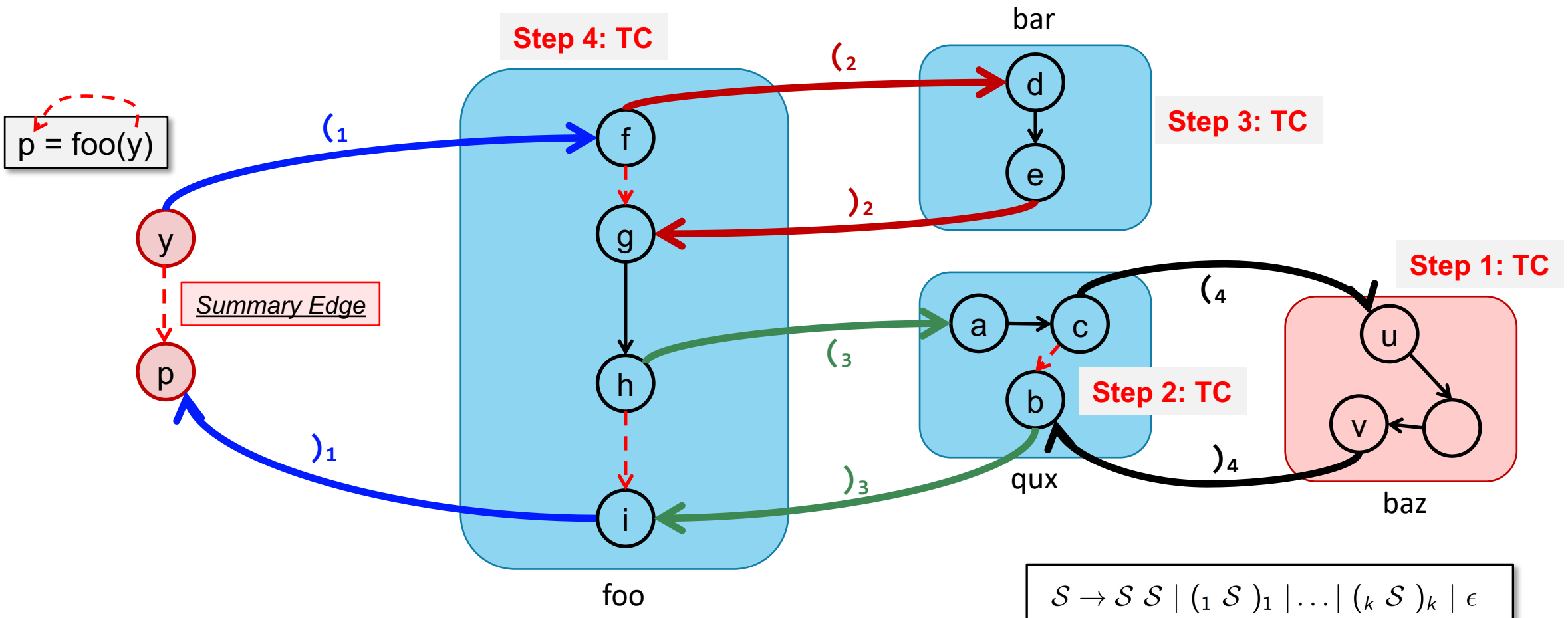
```
: g()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



CS-DDA at the Same Level



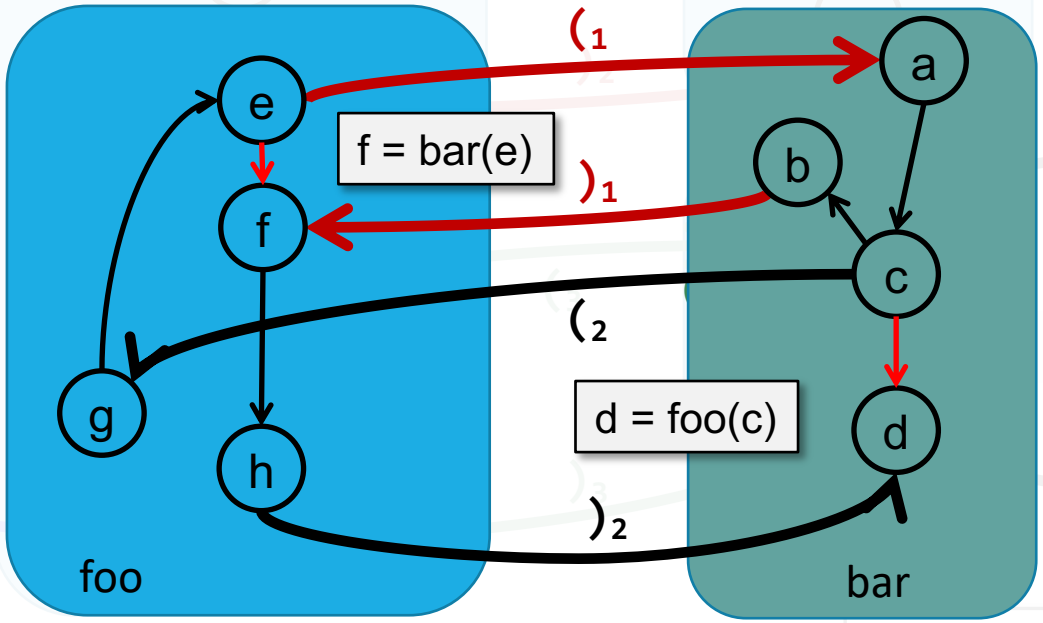
CS-DDA at the Same Level



$$\mathcal{S} \rightarrow \mathcal{S}\ \mathcal{S} \mid (_1\ \mathcal{S}\ )_1 \mid \ldots \mid (_k\ \mathcal{S}\ )_k \mid \epsilon$$

CS-DDA at the Same Level

More expensive if there are recursive function calls!



Step 2: TC

Step 1: TC

Step 3: TC

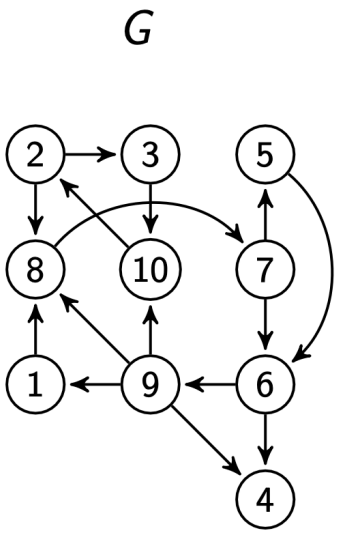
CS-DDA at the Same Level

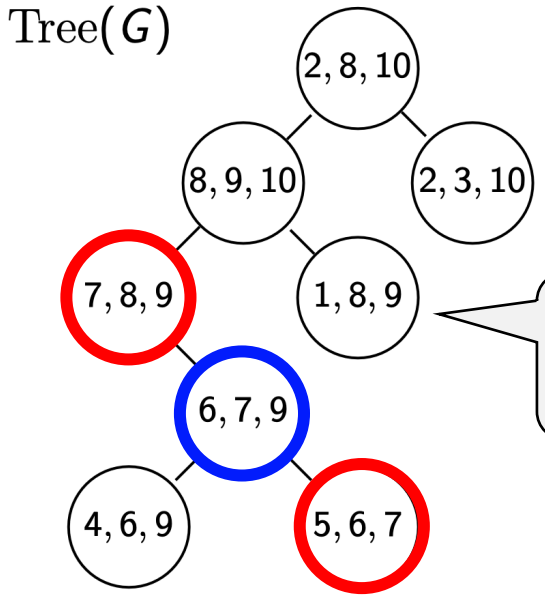
- **Key Operation 1**: Computing TC for a function ~O(n)
- **Key Operation 2**: Updating TC for a function ~O(logn) → ~O(1)

- **Key Observation**: The graph of each function is **sparse**, i.e., has a low treewidth (< 10)

CS-DDA at the Same Level

Definition [Tree Decomposition]
Given a graph G=(V, E), a tree decomposition of the graph Tree(G)=(V_T, E_T) is a tree of bags $B_i \subseteq V$.

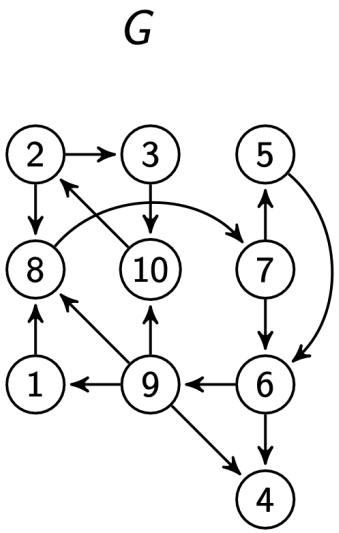


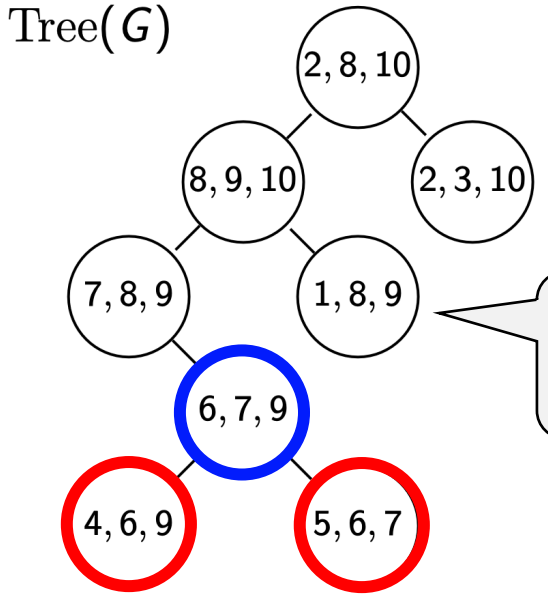


1. If (u, v) ∈ E, there exists a B_i containing u, v.
2. For any simple path: $B_i \rightsquigarrow B_k \rightsquigarrow B_j$, $B_i \cap B_j \subseteq B_k$

CS-DDA at the Same Level

Definition [Tree Decomposition]
Given a graph G=(V, E), a tree decomposition of the graph Tree(G)=(V_T, E_T) is a tree of bags $B_i \subseteq V$.

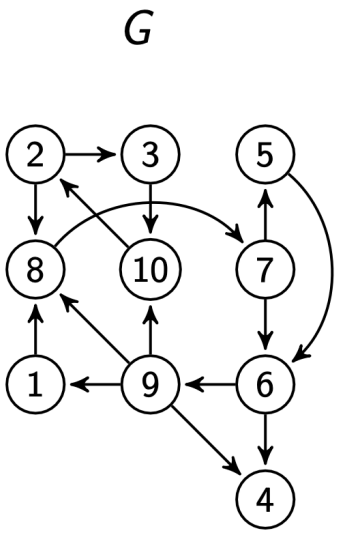


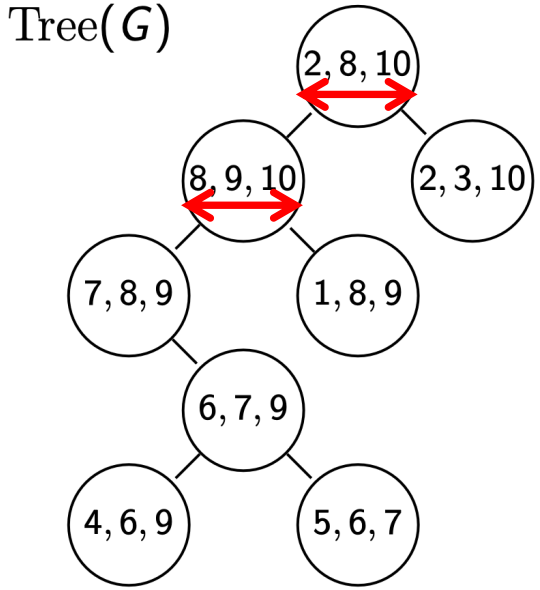


1. If (u, v) ∈ E, there exists a B_i containing u, v.
2. For any simple path: $B_i \rightsquigarrow B_k \rightsquigarrow B_j$, $B_i \cap B_j \subseteq B_k$

CS-DDA at the Same Level

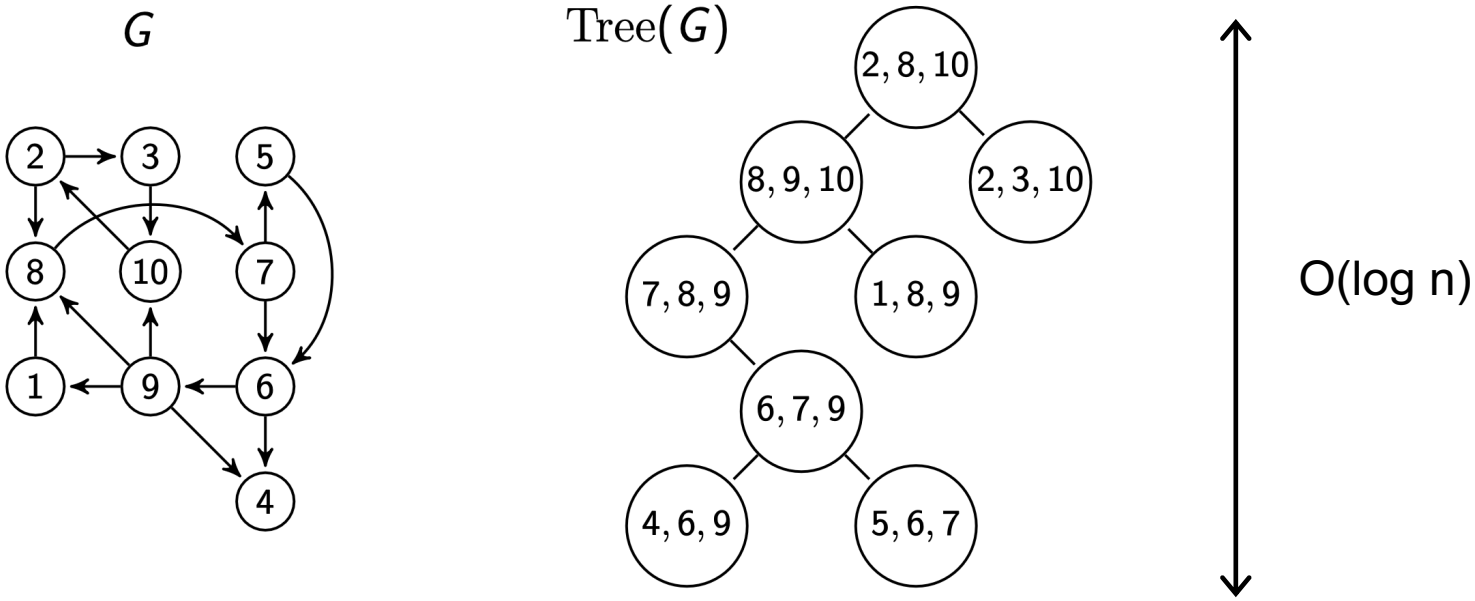
Definition [Treewidth]
The width of Tree(G)=(V_T, E_T) is max|B_i| - 1. The treewidth of a graph G is the minimum width of its tree decompositions.





CS-DDA at the Same Level

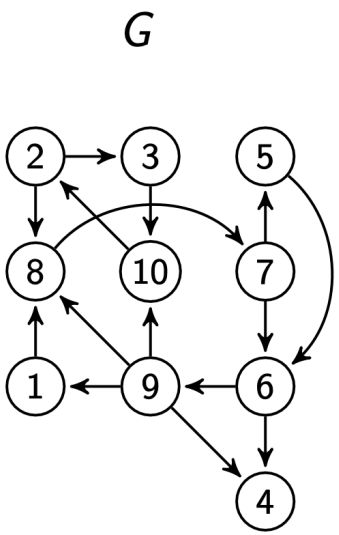
Theorem [Tree Decomposition]
If the treewidth of a graph with n nodes is **O(1)**, Tree(G) can be built in **O(n)** time, and the height of Tree(G) is **O(log n)**.

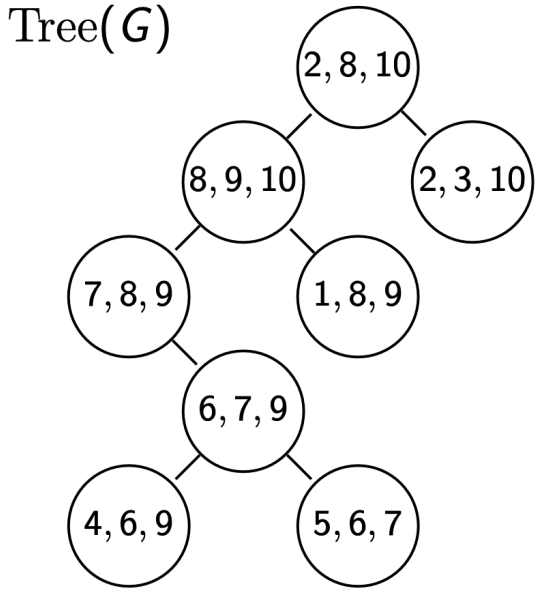


CS-DDA at the Same Level

Theorem [Efficient Reachability Query]
We can answer any reachability query in **O(log n)** time after pre-processing Tree(G) using **O(n)** time and space.

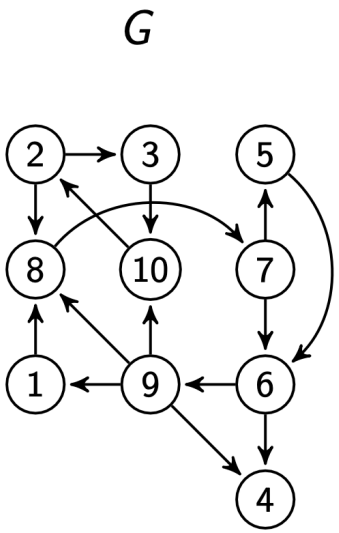
O(1)

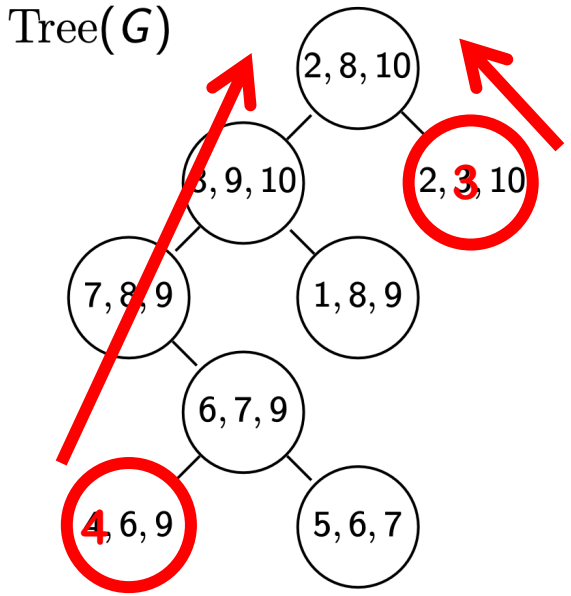




CS-DDA at the Same Level

Theorem [Efficient Reachability Query]
We can answer any reachability query in **O(log n)** time after pre-processing Tree(G) using **O(n)** time and space.





Computing TC for each bag is in O(1)

Computing TC for all bag is in O(n)

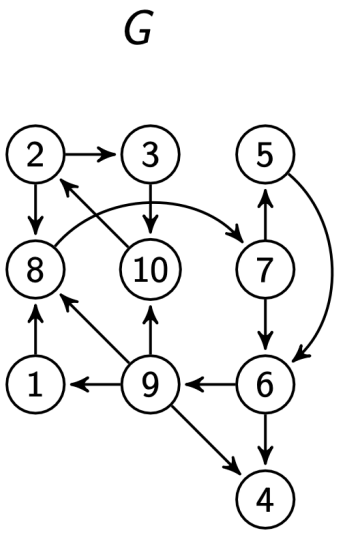
Computing TC for all bag in **bottom-up** order

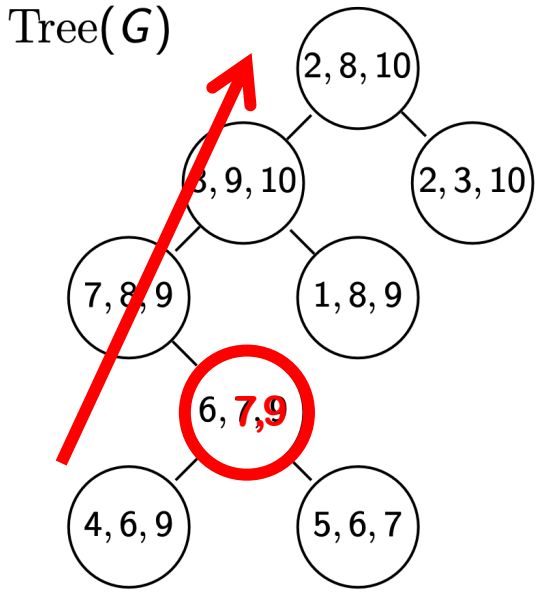
Query(x, y):

Climbing the tree to the top

CS-DDA at the Same Level

Theorem [Efficient Update]
Assume a new edge (x, y) are inserted into G. We can update the reachability computed on Tree(G) in **O(log n)** time if there is a bag B∈Tree(G): x, y ∈ B.





Climbing the tree to update TC

CS-DDA at the Same Level

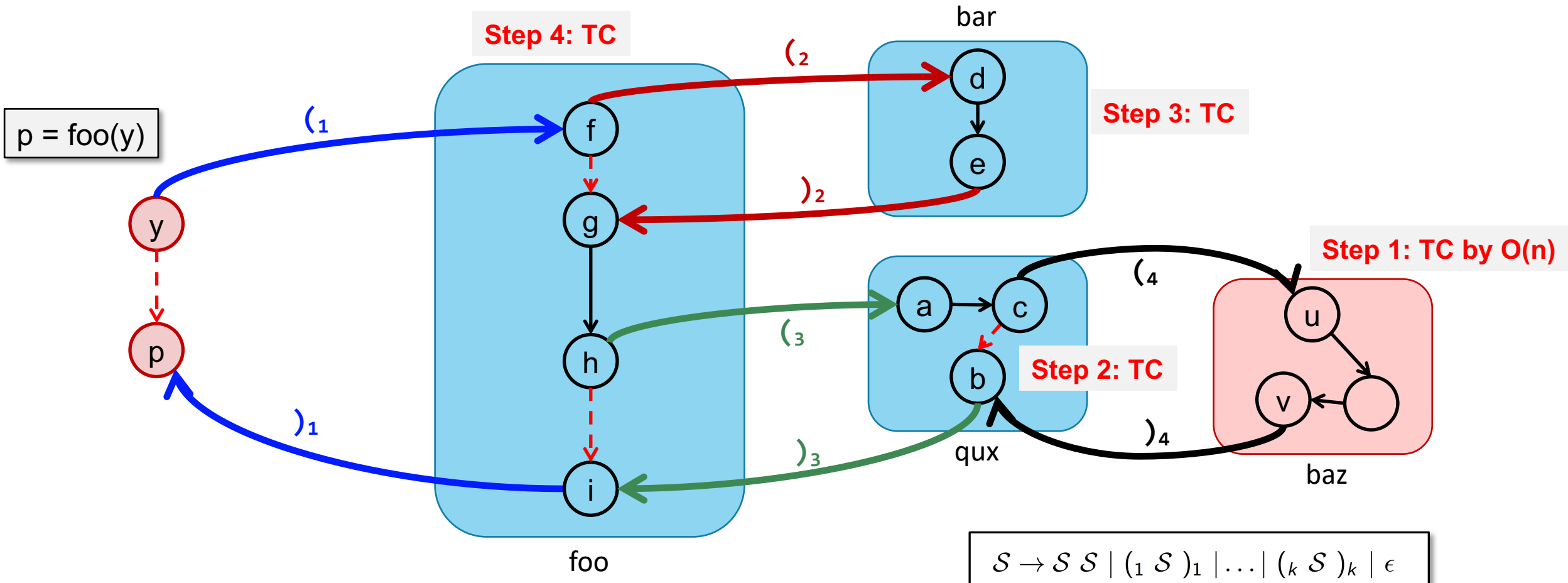
Theorem [Tree Decomposition]
If the treewidth of a graph with n nodes is O(1), Tree(G) can be built in O(n) time, and the height of Tree(G) is **O(log n)**.

Theorem [Efficient Reachability Query]
We can answer any reachability query in **O(log n) (→O(1))** time after pre-processing Tree(G) using O(n) time and space.

Theorem [Efficient Update]
Assume a new edge (x, y) are inserted into G. We can update the reachability computed on Tree(G) in **O(log n)** time if there is a bag B∈Tree(G): x, y ∈ B.

K. Chatterjee, B. Choudhary, and A. Pavlogiannis. Optimal Dyck reachability for data-dependence and alias analysis. POPL 2018.

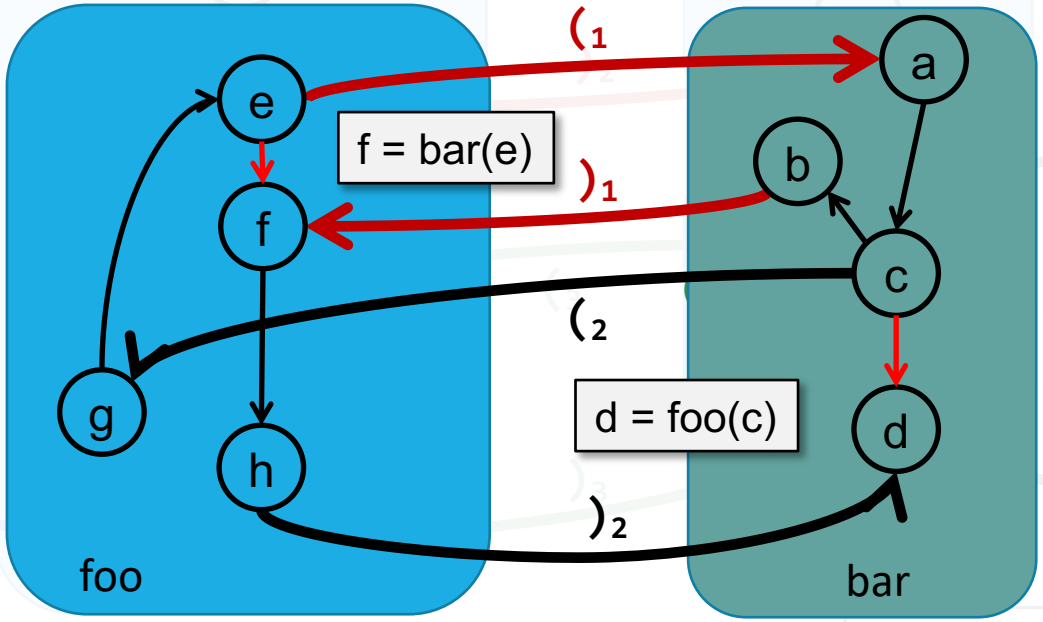
CS-DDA at the Same Level



$$\mathcal{S} \rightarrow \mathcal{S}\ \mathcal{S} \mid (_1\ \mathcal{S}\ )_1 \mid \ldots \mid (_k\ \mathcal{S}\ )_k \mid \epsilon$$

CS-DDA at the Same Level

TC is updated at most $O(a^2k) = O(k)$ times,
where a is # params and returns at each call site, k call site in total
It takes O(n + k•log n) time in total.



Step 2: TC in O(n)

Step 1: TC in O(n)

Step 3: update TC in O(log n)

CS-DDA at the Same Level?

- **Key Operation 1**: Computing TC for a function
- **Key Operation 2**: Updating TC for a function

- **Key Observation**: The graph of each function is sparse with a low treewidth (< 10)

- **Key Result:** Answer any same-level data dependence query in O(1) after a O(n + k*log n) preprocessing

CS-DDA across Different Levels

- Answer any diff-level data dependence query in ~O(1) after ~O(n) preprocessing

: $h()$

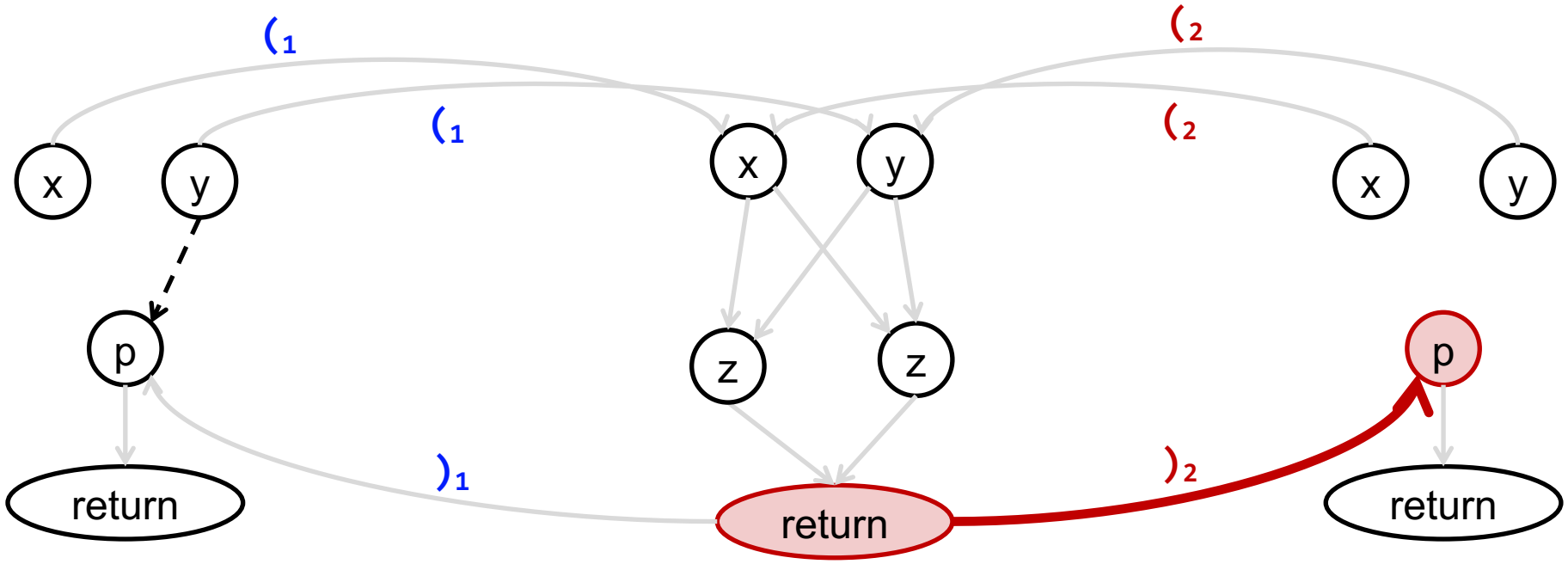
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

: $f(x, y)$

```
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

: $g()$

```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



: $h()$

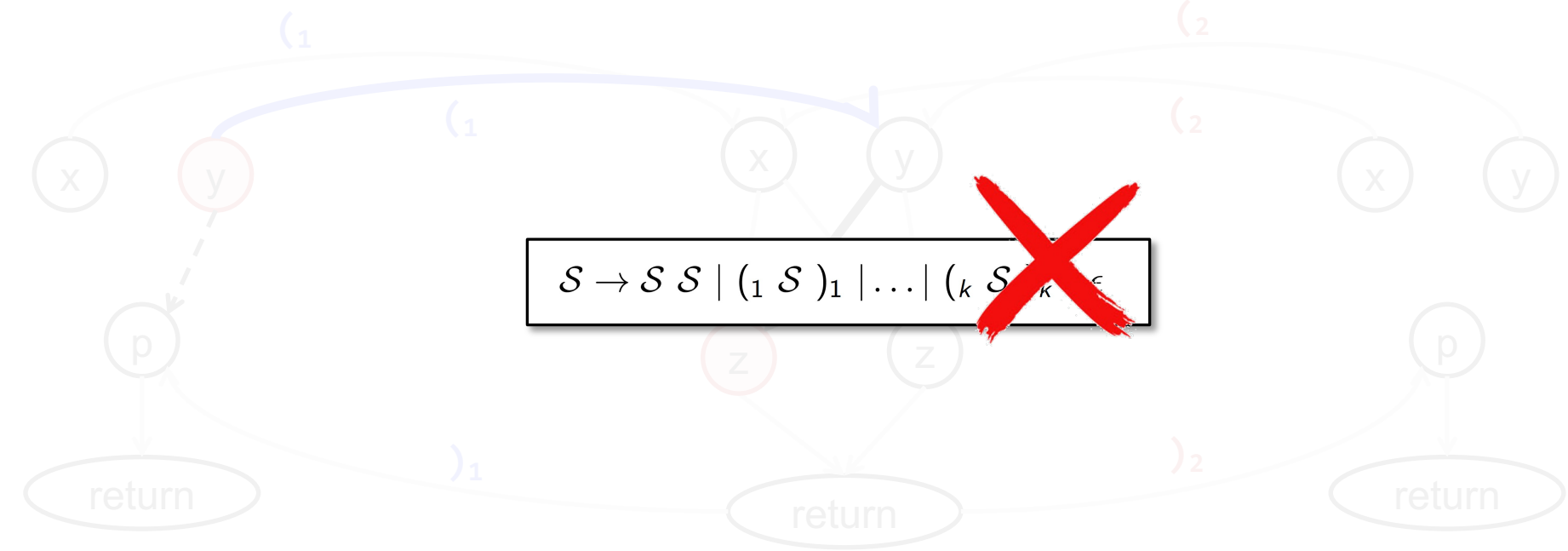
```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

: $f(x, y)$

```
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
5 end
6 return z
```

: $g()$

```
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```



$$\mathcal{S} \to \mathcal{S}\ \mathcal{S} \mid (_1\ \mathcal{S}\ )_1 \mid \ldots \mid (_k\ \mathcal{S}\ )_k \mid \epsilon$$

```
: h()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
4 return p
```

```
: f(x, y)
1 if y%2 = 1 then
2 |  z ← x + y
3 else
4 |  z ← x · y
```

```
: g()
1 x ← 2
2 y ← 2
3 p ← f(x, y)
```

A valid label string can be split into two parts:
1. ⟧ string prefix, with well-matched parentheses, e.g., ⟦⟧, inserted
2. ⟦ string postfix, with well-matched parentheses, e.g., ⟦⟧, inserted

⟧5⟧6⟦1⟦2⟧2⟦3⟦4

⟦1⟦2⟧2⟦3⟦4

$$
\begin{array}{lcl}
S & \rightarrow & P\ N \\
P & \rightarrow & M\ P \mid ⟧_i\ P \mid \epsilon \\
N & \rightarrow & M\ N \mid ⟦_i\ N \mid \epsilon \\
M & \rightarrow & ⟦_i\ M\ ⟧_i \mid M\ M \mid \epsilon
\end{array}
$$

⟦2⟧2⟦3⟧3⟧1⟧4⟧5⟧6

⟦1⟦2⟧2⟦3⟧3⟧1⟦4⟧4

All parentheses must be well-matched → No parentheses are mis-matched

CS-DDA across Different Levels

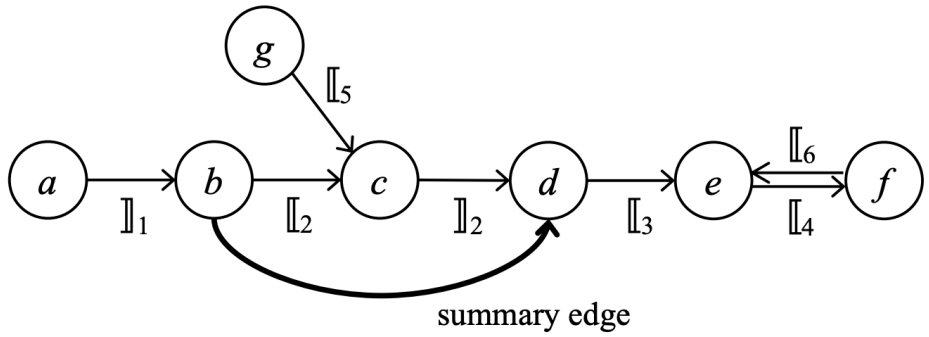
- Answer any diff-level data dependence query in ~O(1) after ~O(n) preprocessing

- **Key Idea:**
  - (1) Use same-level CS-DDA to build summary edges
  - (2) Transform diff-level CS-DDA to conventional reachability
  - (3) Apply reachability indexing techniques

CS-DDA across Different Levels

```
      void bar() {
(1)     int b = foo();
(2)     int d = baz(b);
(3)     qux(d);
      }

      void qux(int e) {
(4)     fax(e);
      }
```

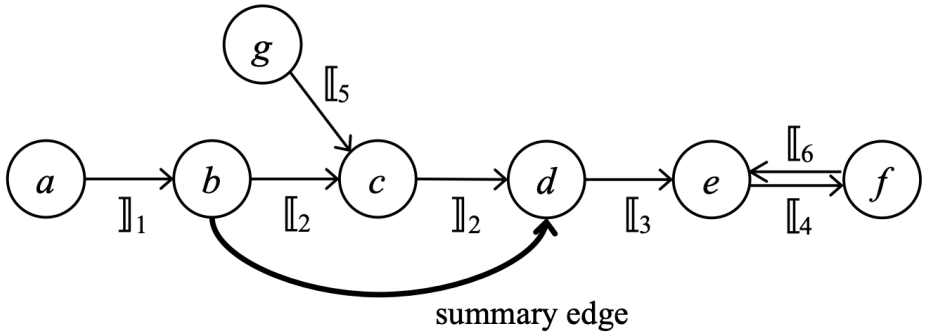
```
      int foo() {
        int g = …; a = …;
(5)     baz(g);
        return a;
      }

      int baz(int c) { return c; }

(6)   void fax(int f) { qux(f); }
```

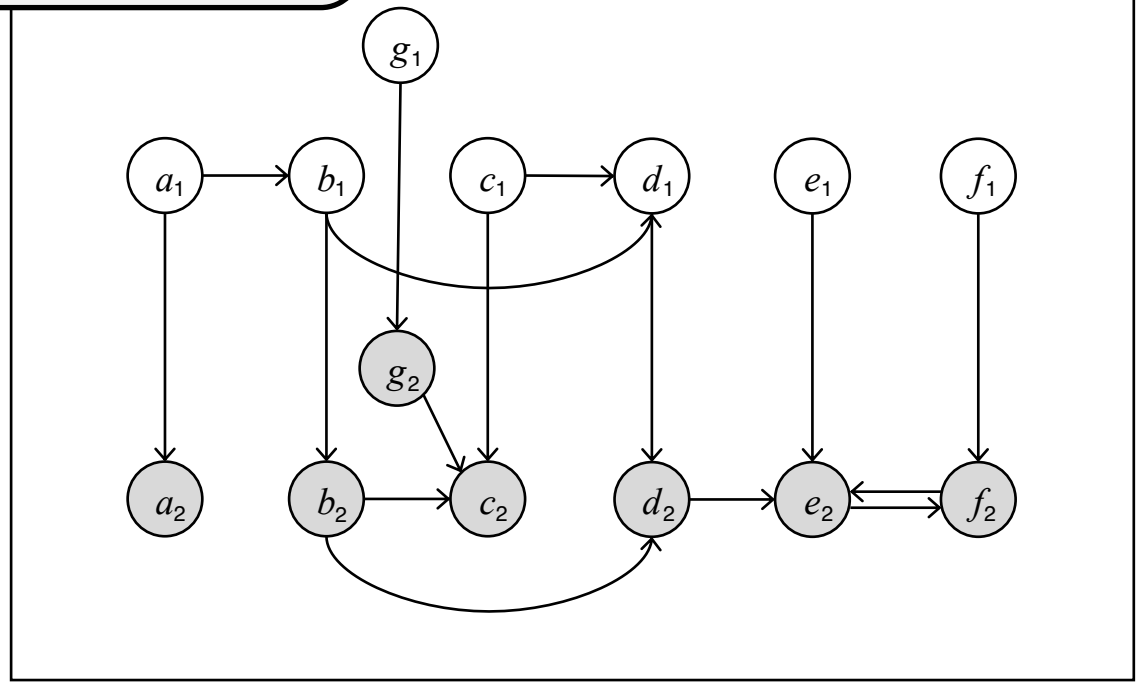


CS-DDA across Different Levels

A valid label string can be split into two parts:
1. ⟧ string prefix, with well-matched parentheses, e.g., ⟦⟧, inserted
2. ⟦ string postfix, with well-matched parentheses, e.g., ⟦⟧, inserted

```
}                                   }
    void qux(int e) {                   int baz(int c) { return c; }
(4)     fax(e);
    }                                   (6) void fax(int f) { qux(f); }
```

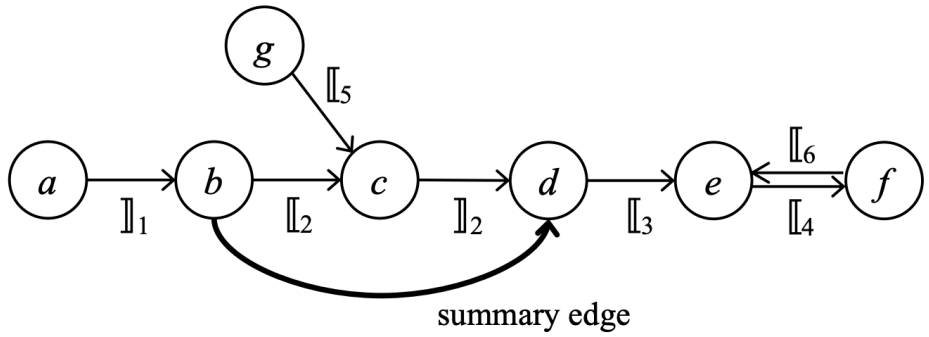




CS-DDA across Different Levels

```
      void bar() {
(1)     int b = foo();
(2)     int d = baz(b);
(3)     qux(d);
      }

      void qux(int e) {
(4)     fax(e);
      }
```

```
      int foo() {
        int g = …; a = …;
(5)     baz(g);
        return a;
      }

      int baz(int c) { return c; }

(6)   void fax(int f) { qux(f); }
```



can a cfl-reach f? => can a_1 reach f_2?